

电子科技大学

UNIVERSITY OF ELECTRONIC SCIENCE AND TECHNOLOGY OF CHINA

学士学位论文

BACHELOR THESIS



论文题目 基于 LLM 的 Shell 机器人

学科专业 计算机科学与技术

学 号 2022060909001

作者姓名 陈嘉乔

指导老师 聂晓文 副教授

学 院 计算机科学与工程学院（网络空间安全学院）

摘 要

随着大语言模型（Large Language Model, LLM）能力提升，自然语言驱动的 Shell 助手成为降低命令行门槛的重要方向。但在中文运维场景中，系统仍需同时解决知识覆盖、生成准确性与执行安全三类问题。

本文围绕“基于 LLM 的 Shell 机器人”，设计并实现一套检索增强与安全执行一体化系统。系统采用客户端-服务端分离架构：服务端以本地量化 Qwen-7B 为生成核心，客户端负责会话编排、分步执行与用户确认；并通过 SQLite 构建短期、长期、摘要三层记忆机制，支持多轮上下文保持与多会话管理。

在知识与检索层，本文构建覆盖 45 篇文档、88 个文本块的中文 Shell 知识库，并采用“向量检索 + 关键词信号”的混合检索与加权重排策略。在 80 条自然语言查询集上，最优配置达到 SrcHit=0.975、MRR=0.9375、nDCG=0.9105、Recall@1=0.900，相较纯向量检索基线在核心排序指标上取得稳定提升。

在生成与执行层，本文实现“语法校验-风险分级-效果模拟”预执行流水线，结合权限控制与结构化审计日志形成安全闭环。在 50 条端到端生成评测样本上，系统达到语法正确率 92.0%，语义高/中匹配占比 78.0%，沙箱预执行成功率 54.0%，并对 40.0% 高风险样本进行主动拦截，体现了可用性与安全性的协同设计目标。

实验结果表明，本文方法在中文 Shell 命令生成任务上展示了初步的工程可行性，能够为常见运维场景提供可检索、可解释、可控执行的本地化智能体方案。

关键词：大语言模型，Shell 智能体，检索增强生成，混合检索，命令安全执行

ABSTRACT

Large Language Models (LLMs) enable natural-language interfaces for command-line tools, but Chinese Shell assistance still requires a practical balance among retrieval quality, command correctness, and execution safety. This thesis presents an LLM-based Shell agent that integrates Retrieval-Augmented Generation (RAG) with a guarded execution pipeline in a local deployment setting.

The system adopts a client-server architecture. The server hosts a locally quantized Qwen-7B model, while the client handles session orchestration, user confirmation, and stepwise execution. A three-layer memory design (short-term, long-term, and summary memory) is implemented with SQLite to preserve multi-turn context across sessions.

For knowledge grounding, we build a Chinese Shell knowledge base with 45 documents and 88 text chunks, and propose a hybrid retrieval strategy that combines dense vector similarity with keyword-aware weighted reranking. On an 80-query natural-language benchmark, the best configuration achieves $\text{SrcHit} = 0.975$, $\text{MRR} = 0.9375$, $\text{nDCG} = 0.9105$, and $\text{Recall}@1 = 0.900$, outperforming the pure vector baseline on major ranking metrics.

For safety-aware generation, we design a pre-execution chain of syntax validation, risk grading, and effect simulation, together with permission checks and structured audit logging. In end-to-end generation evaluation on 50 representative queries, the system obtains 92.0% syntax accuracy, 78.0% high-or-medium semantic match rate, and 54.0% sandbox dry-run success rate, while proactively blocking 40.0% high-risk outputs.

Results indicate that the proposed approach is engineering-ready for common Chinese Shell operation scenarios, providing a controllable and auditable path from natural-language intent to executable commands.

Keywords: Large Language Model, Shell Agent, Retrieval-Augmented Generation, Hybrid Retrieval, Command Safety

目 录

第一章 绪 论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	1
1.2.1 自然语言到代码生成与 LLM Agent	1
1.2.2 检索增强生成技术	2
1.3 研究内容与贡献	3
1.4 论文组织结构	3
1.5 本章小结	4
第二章 相关背景	5
2.1 检索增强生成	5
2.1.1 基本原理	5
2.1.2 RAG 系统的基本组成	5
2.1.3 RAG 技术的发展脉络	5
2.2 向量检索与嵌入模型	6
2.2.1 稠密向量检索	6
2.2.2 ChromaDB 向量数据库	6
2.2.3 文档分块策略	7
2.3 混合检索与重排序	7
2.3.1 稀疏检索的互补价值	7
2.3.2 混合检索的实现方式	7
2.3.3 重排序流程	7
2.4 Shell 命令安全	8
2.4.1 Shell 命令执行的风险特征	8
2.4.2 命令安全的防护层次	8
2.5 大语言模型基础与 In-Context Learning	8
2.5.1 大语言模型的涌现能力	8
2.5.2 少样本学习与提示工程	9
2.5.3 指令微调与参数高效微调	9
2.6 LLM Agent 与自主推理系统	9
2.6.1 Agent 系统的框架	9

2.6.2 操作系统级 Agent	10
2.7 模型参数高效训练与量化	10
2.7.1 大模型量化的必要性	10
2.7.2 NF4 量化原理	10
2.7.3 双重量化与本系统配置	11
2.8 本章小结	11
第三章 系统架构	12
3.1 整体设计	12
3.1.1 设计目标	12
3.1.2 总体架构	12
3.2 CLI 客户端模块	13
3.3 模型服务器模块	16
3.3.1 LLM 配置	16
3.3.2 提示模板	16
3.3.3 会话管理	17
3.4 RAG 检索模块	17
3.5 内存管理模块	17
3.6 系统端到端处理流程	18
3.7 本章小结	18
第四章 领域 RAG 检索系统	19
4.1 知识库构建	19
4.1.1 文档分类体系	19
4.1.2 文档增强策略	19
4.1.3 索引构建	20
4.1.4 评估驱动的迭代扩库方法论	20
4.2 嵌入与索引	22
4.2.1 中文嵌入模型选型	22
4.2.2 分块策略与参数	22
4.3 混合检索与重排策略	22
4.3.1 向量检索候选集构建	22
4.3.2 关键词候选集构建	23
4.3.3 合并与重排	23
4.4 关键词路由过滤	23

4.4.1 路由机制设计	23
4.4.2 路由规则.....	23
4.4.3 路由的保底策略	24
4.5 本章小结	24
第五章 命令安全执行系统	25
5.1 设计目标.....	25
5.2 三级权限分级机制	25
5.2.1 风险等级定义.....	25
5.2.2 分级判定规则.....	25
5.2.3 典型命令分类样例	26
5.2.4 规则链的边界情况与设计权衡	26
5.3 预执行校验流水线	28
5.3.1 三步流水线设计	28
5.4 执行日志系统	30
5.4.1 日志结构.....	30
5.4.2 CSV 导出	30
5.4.3 测试覆盖.....	30
5.5 本章小结	31
第六章 实验结果与分析.....	32
6.1 实验设置.....	32
6.1.1 硬件与软件环境	32
6.1.2 知识库构建	32
6.1.3 评估数据集	32
6.1.4 实验方案设计	33
6.1.5 实验流程与评估协议.....	33
6.2 评估指标.....	34
6.2.1 来源命中率 (Source Hit Rate)	34
6.2.2 关键词命中率 (Keyword Hit Rate)	34
6.2.3 平均倒数排名 (MRR)	34
6.2.4 归一化折损累积增益 (nDCG)	35
6.2.5 召回率 @k (Recall@k)	35
6.2.6 指标解释与组合使用原则	35
6.3 实验结果.....	35

6.3.1 A/B 检索策略对比（无路由，最终知识库）	35
6.3.2 关键词路由消融实验.....	36
6.3.3 知识库规模演进对比.....	37
6.3.4 开发集模板查询诊断结果（非最终评估）	37
6.3.5 命令生成质量评估	38
6.4 分析与讨论	39
6.4.1 混合检索优于单一向量检索的原因	39
6.4.2 关键词路由性能下降的成因分析.....	39
6.4.3 评估指标间的相关性.....	40
6.4.4 与相关工作的对比分析	40
6.4.5 系统整体性能评估	41
6.4.6 性能与延迟分析	42
6.4.7 失败案例与边界情况分析.....	43
6.4.8 实验有效性与威胁分析	44
6.5 本章小结.....	44
第七章 总结与展望	46
7.1 工作总结.....	46
7.2 创新点	46
7.3 未来展望.....	47
7.4 本章小结.....	48
致 谢	49

第一章 绪论

1.1 研究背景与意义

Shell（如 Bash、Zsh）是 Linux/Unix 操作系统的核心交互工具，用户通过命令行直接控制文件系统、进程管理、网络通信等底层功能。然而，Shell 命令语法复杂，涉及路径规则、参数组合、管道与重定向等专业知识，对非技术背景用户存在较高的使用门槛，高危命令（如 `rm -rf /`、`dd of=/dev/sda`）的误执行可造成不可逆的数据损失。

近年来，以 GPT 系列、LLaMA、Qwen^[1] 为代表的大语言模型在自然语言理解与代码生成领域取得了显著突破。将 LLM 能力与 Shell 操作结合，构建自然语言驱动的 Shell 命令生成系统，是降低操作系统使用门槛、提升运维效率的重要研究方向。用户只需以中文口语化描述操作意图（如“查看当前目录下超过 100MB 的文件”），系统即可自动生成对应的 Shell 命令（如 `find ./ -size +100M`）并辅助执行。

然而，通用 LLM 直接应用于 Shell 场景面临三项核心挑战：其一，LLM 训练数据以英文为主，对中文口语化表达的理解存在偏差；其二，LLM 训练语料中 Shell 领域长尾知识（特定发行版命令、企业内部操作流程）分布稀疏，导致模型易产生语法正确但语义错误的命令（即“幻觉”问题）；其三，Shell 命令执行具有强副作用，安全性保障是部署 LLM 辅助系统的前提条件。

基于上述背景，本课题围绕中文 Shell 智能体的可靠性与安全性展开研究，具有重要的理论意义与实践价值。

1.2 国内外研究现状

1.2.1 自然语言到代码生成与 LLM Agent

自然语言到代码生成（Natural Language to Code, NL2Code）是人工智能领域的经典研究课题。早期工作基于统计机器翻译方法将自然语言映射到程序语言，近年来基于 Transformer 架构的预训练语言模型大幅提升了代码生成的准确率。Codex^[2]、Code LLaMA^[3] 等专用代码生成模型的发布标志着该领域的重要进展。Shell 命令生成作为 NL2Code 的子方向^[2]，具有命令结构简短、参数组合多样、中文口语表达形式不固定等特点，对模型的语义理解能力提出了更高要求。

同时，LLM 在推理与工具使用中的能力已成为新的研究焦点。ReAct 框架^[2]通过交织推理步骤（Thought）与行动步骤（Action）的方式，显著提升了 LLM 在

复杂任务中的成功率。Toolformer^[2]证实了大语言模型能够自学使用 API 和工具。OS-Copilot^[2] 针对操作系统级任务构建了通用的计算机 Agent 框架，为本文的系统设计提供了重要参考。AgentBench^[2] 等评估框架的推出为 Agent 系统的性能衡量提供了标准化指标。

1.2.2 检索增强生成技术

检索增强生成（Retrieval-Augmented Generation, RAG）通过在推理时动态检索外部知识库并将相关文档注入提示上下文，有效缓解了 LLM 的知识局限与幻觉问题。RAG 技术已在开放域问答、文档摘要、代码补全等多个场景中得到广泛验证^[2,3]。Self-RAG^[2] 进一步引入了检索反思机制，使模型能够动态决定何时检索。向量检索（基于稠密嵌入的余弦相似度匹配）与稀疏检索（基于 BM25 等词频统计）的混合策略被证明在多个评测基准上优于单一检索方式^[2]，能够在语义相似度与关键词精确匹配之间实现互补。

LLM 生成内容的安全性已成为系统部署的重要考量。在 Shell 命令执行场景下，提示注入攻击（Prompt Injection）^[2]、危险命令生成等风险需通过系统级防护机制加以规避。现有工作主要从输出过滤、提示设计和权限控制三个维度进行防护，但针对 Shell 命令预执行模拟校验的系统性方案仍相对欠缺。

针对 LLM 对齐与安全，通过人类反馈的强化学习（RLHF）^[2] 已成为主流方案。深度强化学习从人类偏好^[2] 出发，指导模型学习人类价值观。最近的工作从提示设计与输出过滤角度探索了针对 LLM 集成应用的安全防护方案。

工具学习是扩展 LLM 能力的重要方向。Tool Augmented Language Models (TALM)^[2] 使模型能够学习何时调用工具。API-Bank^[2] 基准为工具调用能力提供了系统化评估。

链式思维提示（Chain-of-Thought）^[2] 通过多步推理显著提升了 LLM 的性能。大语言模型作为零样本推理器^[2] 展示了其强大的泛化能力。涌现能力分析^[2] 揭示了 LLM 在模型规模增大时的非线性性能跳跃在 Shell 命令执行场景下，提示注入攻击（Prompt Injection）^[2]、危险命令生成等风险需通过系统级防护机制加以规避。现有工作主要从输出过滤、提示设计和权限控制三个维度进行防护，但针对 Shell 命令预执行模拟校验的系统性方案仍相对欠缺。

综上，现有工作尚未针对中文 Shell 场景提供一个同时解决领域知识覆盖不足、中文语义理解偏差与命令执行安全三项挑战的端到端系统。本文的研究正是针对这一空白展开。

1.3 研究内容与贡献

本文的主要研究内容与贡献如下：

1. 设计并实现了基于 LLM 的中文 Shell 智能体系统。采用客户端-服务端分离架构，以本地量化 Qwen-7B 模型为推理核心，实现自然语言描述到 Shell 命令的端到端生成，并适配 Bash 与 Zsh 两种主流 Shell 的语法差异。系统支持多会话管理机制，允许用户在同一进程中管理多个独立的任务上下文，每个会话拥有唯一的标识符与隔离的对话记忆，实现了用户在不同工作流之间快速切换的能力。
2. 构建了面向 Shell 领域的 RAG 检索模块。以 ChromaDB 向量数据库与 BAAI/bge-small-zh-v1.5 中文嵌入模型为基础，提出混合检索与关键词路由策略，在 80 个自然语言查询评估集上将 Recall@1 提升至 0.90、MRR 提升至 0.9375。
3. 设计并实现了三级命令安全执行框架。包含基于白名单/黑名单的权限分级机制、预执行“语法校验—风险评估—效果模拟”三步流水线及结构化执行日志系统，通过 58 个单元测试与集成测试用例验证其正确性。
4. 构建了分层对话记忆系统。基于 SQLite 实现短期记忆、长期记忆与摘要记忆三层架构，支持多轮对话的上下文关联，使系统能够理解如“将这些文件移动到/data/backup”此类依赖历史上下文的后续指令。
5. 实现了面向复杂任务的分步执行机制。基于关键词启发式规则自动检测多步骤任务请求，通过逐步生成—安全校验—用户确认—工作记忆更新的循环，驱动复杂任务的连续执行，支持最多 12 步操作序列，每步均享有独立的安全保障与可中断能力。

1.4 论文组织结构

本文其余部分组织如下：第二章介绍与本研究相关的背景知识，包括检索增强生成技术、向量检索与嵌入模型、混合检索与重排序方法及 Shell 命令安全机制；第三章描述系统整体架构及各核心模块的设计，重点阐述 CLI 客户端的多会话管理机制、响应解析容错策略、分步任务执行流程及模型服务器的会话记忆管理；第四章详述领域 RAG 检索系统的知识库构建、嵌入索引、混合检索策略与关键词路由过滤；第五章介绍命令安全执行系统的权限分级模型、预执行校验流水线及执行日志系统；第六章展示实验设置、评估指标与消融实验结果，并进行分析与讨论；第七章总结全文主要工作与创新点，并对未来工作进行展望。

1.5 本章小结

本章阐述了以自然语言驱动 Shell 命令生成的研究背景与意义，指出通用 LLM 在中文 Shell 场景下面临的中文语义理解偏差、领域知识覆盖不足与命令执行安全三项核心挑战，回顾了自然语言到代码生成、检索增强生成及 LLM 安全领域的相关研究现状，并明确了本文的研究内容与主要贡献，包括多会话隔离机制、混合检索策略、分步执行框架等创新点。后续各章将围绕系统架构设计、多会话管理、分层记忆系统、RAG 检索策略、命令安全执行与实验验证逐步展开。

第二章 相关背景

2.1 检索增强生成

2.1.1 基本原理

检索增强生成（Retrieval-Augmented Generation, RAG）是一种将外部知识检索与大语言模型生成相结合的技术框架。其核心思想是：在推理阶段，先从外部知识库中检索与用户查询最相关的文档片段，再将这些片段作为上下文注入提示（Prompt），由语言模型基于上下文生成最终回复。

与直接依赖模型参数内隐知识的生成方式相比，RAG 具有以下优势：第一，模型无需记忆全部领域知识，知识库可独立更新而无需重新训练模型；第二，通过控制检索到的文档质量，可有效降低模型幻觉的发生概率；第三，检索来源可追溯，有助于提升系统输出的可解释性。

2.1.2 RAG 系统的基本组成

一个典型的 RAG 系统由以下三个模块组成：

1. 知识库构建模块：将原始文档进行分块（Chunking）、编码（Encoding）并存储至向量数据库，形成可检索的索引。分块策略（大小、重叠长度）对检索质量有重要影响。
2. 检索模块：接收用户查询，将其编码为向量，在向量数据库中查找语义相近的文档块，并按相关性排序返回 Top- k 结果。
3. 生成模块：将检索到的文档块拼接至提示模板，调用大语言模型生成最终回复。

2.1.3 RAG 技术的发展脉络

RAG 技术的研究起源于对大语言模型知识局限性的系统性认识。早期的语言模型依赖固定参数内隐知识，一旦训练完成便无法获取新知识，在时效性强或高度专业化的问答场景中表现明显不足。Lewis 等人在 2020 年提出的原始 RAG 框架将文档检索器与序列生成模型端到端联合训练，首次验证了“检索 + 生成”范式在开放域问答任务上的有效性。

此后，RAG 研究沿两条主线演进：其一是检索精度提升，通过双编码器微调、交叉编码器重排序^[1]、迭代检索等手段，使检索模块返回的上下文文档与查询意图更加吻合；其二是系统架构优化，从最初的一次性检索扩展至多步推理中的多

轮检索，并引入检索必要性判断（即“是否应检索”的路由决策），降低无关文档引入的噪声。

面向专业领域的 RAG 系统还需额外应对词汇鸿沟与知识库覆盖不足两个问题^[2]。词汇鸿沟指用户口语化查询与专业文档之间的表述差异，覆盖不足则指领域知识库规模有限时检索召回率低的问题。面向领域的 RAG 工程实践通常通过同义问法注入、失败查询覆盖等启发式增强手段缓解词汇鸿沟问题，并以评估驱动的迭代流程持续改善知识库质量，这已成为当前领域 RAG 系统构建的主流范式之一。

2.2 向量检索与嵌入模型

2.2.1 稠密向量检索

稠密向量检索（Dense Retrieval）是 RAG 系统的核心检索机制。其基本流程为：利用嵌入模型（Embedding Model）将文本映射至高维稠密向量空间，查询文本与文档文本在同一向量空间中通过余弦相似度（Cosine Similarity）衡量语义相关性。对于查询向量 q 与文档向量 d ，余弦相似度定义为：

$$\text{sim}(q, d) = \frac{q \cdot d}{\|q\| \cdot \|d\|} \quad (2-1)$$

经归一化后，向量数据库中存储的余弦距离 $dist$ 与相似度分数之间满足：

$$\text{score}_{\text{vec}} = \frac{1}{1 + dist} \quad (2-2)$$

面向中文场景的嵌入模型需在中文语义相似度任务上表现优异，同时参数量适中以支持本地部署。BAAI 发布的 bge 系列模型^[2]针对中文文本优化，在多个中文语义基准上取得较好结果，是中文 RAG 系统的常见选择之一。

2.2.2 ChromaDB 向量数据库

向量数据库是 RAG 系统持久化存储文档向量的核心组件，通常支持向量相似度查询、元数据过滤与集合管理等功能。ChromaDB 是一款开源的本地向量数据库，支持持久化存储与元数据过滤，可在无网络依赖的环境中运行，适合隐私敏感或离线部署场景。元数据过滤能力使系统在检索时能够将候选范围限制在特定类别文档中，从而提升检索精度。

2.2.3 文档分块策略

一种常见的领域文档分块策略是 Markdown 感知的自适应分块：优先按 Markdown 标题和双换行符切分段落以保留语义边界，在段落粒度内控制块大小和重叠长度，对超长段落回退至字符级滑动窗口切分。此策略能够保留文档的语义结构，避免将同一语义单元割裂到不同块中。

2.3 混合检索与重排序

2.3.1 稀疏检索的互补价值

纯向量检索在语义相似度匹配上表现良好，但对精确术语（如命令名称 `tar`、`chmod` 等）的检索存在不足：若查询中出现罕见词汇而嵌入模型对其语义泛化处理，可能导致语义正确但关键词不匹配的文档排名靠后。稀疏检索方法（以 BM25 为代表^[7]）基于词频统计，在精确关键词匹配上具有天然优势，与向量检索形成互补。本系统即采用此轻量方案：以查询词覆盖率（即命中词数与查询词数之比）作为词法相关性评分，无需维护全局词频统计，实现简单，在小规模领域知识库上仍能提供有效的关键词补强信号。

2.3.2 混合检索的实现方式

混合检索的一种常见实现方式是以加权乘积将向量相似度与关键词命中率合并，引入关键词命中率作为辅助信号对候选文档进行重排。设文档 d 的向量相似度分数为 $s_{\text{vec}}(d)$ ，关键词命中率为 $s_{\text{kw}}(d)$ ，则混合重排分数可定义为：

$$\text{score}(d) = s_{\text{vec}}(d) \times (1.0 + \alpha \cdot s_{\text{kw}}(d)) \quad (2-3)$$

其中 α 为关键词权重系数，通过在评估集上调参确定。乘积形式保证了向量分数低的文档即使关键词完全命中也不会被过度提升，从而在语义相关性与关键词精确性之间取得平衡。此外，为防止低质量关键词候选引入噪声，通常设置命中率阈值对候选集进行过滤。

2.3.3 重排序流程

混合检索重排序的完整流程如下：

1. 分别构建向量候选集与关键词候选集；
2. 以文档块 ID 为键合并两个候选集，向量候选已有记录时直接补充其关键词分数，纯关键词候选通过命中率阈值过滤后加入合并集合；

3. 按混合分数降序排列，截取 Top- k 返回。

2.4 Shell 命令安全

2.4.1 Shell 命令执行的风险特征

Shell 命令执行具有以下风险特征，使其成为 LLM 辅助系统的高风险组件：

1. 强副作用性：文件删除（`rm`）、权限变更（`chmod`）、磁盘写操作（`dd`）等命令一旦执行，影响往往不可逆；
2. 组合爆炸性：Shell 命令通过管道、重定向等操作符组合，单个操作符的语义改变可能导致截然不同的执行结果；
3. 提示注入风险：若用户输入中包含精心构造的自然语言，可能诱导 LLM 生成与意图不符甚至危险的命令（即提示注入攻击，Prompt Injection）。

2.4.2 命令安全的防护层次

针对上述风险，业界通常从以下几个层次构建 Shell 命令安全防护：

输出过滤层：在 LLM 生成命令后，通过规则引擎或分类模型检测危险模式，拦截高风险命令。

权限控制层：基于白名单/黑名单机制，明确定义允许执行的命令范围，对超出范围的命令予以拒绝。

预执行模拟层：在实际执行前以只读或模拟方式预览命令效果，使用户能够在确认结果后再决定是否执行。

本系统的安全框架覆盖以上全部三个层次，具体实现详见第五章。

2.5 大语言模型基础与 In-Context Learning

2.5.1 大语言模型的涌现能力

近年来，基于 Transformer 架构的大语言模型（LLM）的规模扩展（Scale）呈现出一个显著的非线性效应。涌现能力^[7]研究表明，当模型规模突破某个临界点时，模型会突然获得在小规模模型上完全不存在的复杂推理能力。这些能力包括多步数学推理、代码生成、常识推理等高阶认知任务，但并未在模型的任何中间阶段表现出来。

LLaMA 系列与 Qwen 系列等开源大模型的发布使得学术界和工业界能够在消费级硬件上进行大规模模型研究，推动了 LLM 应用的快速发展。GPT-4 技术报告^[7]进一步提升了大模型性能的整体水平。

2.5.2 少样本学习与提示工程

大语言模型在没有任何训练数据的情况下就能执行特定任务的能力被称为零样本学习（Zero-Shot Learning）。更进一步，通过在提示中提供少量示例的方式，模型的任务性能会显著提升，这被称为少样本学习（Few-Shot Learning）^[2]。In-Context Learning 是这一现象的核心机制：模型从提示中的上下文学习到目标任务的模式，而无需显式的参数更新。

链式思维（Chain-of-Thought）提示^[2]通过让模型逐步展示推理过程，显著提升了模型在复杂推理任务上的成功率。大语言模型作为零样本推理器^[2]进一步验证了这一思路的有效性。提示工程的系统性综述^[2]总结了预训练、提示与预测范式在自然语言处理中的应用。

2.5.3 指令微调与参数高效微调

虽然 In-Context Learning 强大，但针对特定领域任务的精调（Fine-tuning）仍能提供进一步的性能收益。指令微调（Instruction Tuning）^[2]通过在多样化指令数据上进行微调，使通用模型能够更好地理解和遵循用户指令。

当模型规模足够大时，全参数微调的计算开销往往难以承受。LoRA（Low-Rank Adaptation）^[2]通过在权重矩阵中插入低秩可训练矩阵，显著降低了微调过程中的可训练参数数量。QLoRA^[2]进一步结合 4-bit 量化技术，使得在消费级 GPU 上进行大模型微调成为可能。

2.6 LLM Agent 与自主推理系统

2.6.1 Agent 系统的框架

LLM Agent 是指由大语言模型作为决策引擎的自主系统。与仅执行单轮推理的 LLM 不同，Agent 在推理过程中能够：(1) 规划：制定实现目标的多步骤计划；(2) 行动：调用工具或执行指令；(3) 反思：评估结果并调整后续计划。

ReAct 框架^[2]通过让模型交织生成推理（Thought）和行动（Action）步骤，实现了这种循环推理-行动范式。与仅进行推理或仅进行行动的方案相比，ReAct 在多项任务上都展现了更强的性能。

Reflexion^[2]更进一步，通过长期记忆（episodic memory）和自我反思机制，使 Agent 能够从失败中学习。

2.6.2 操作系统级 Agent

OS-Copilot^[7] 针对操作系统交互任务构建了端到端的 Agent 系统。该系统能够理解自然语言指令，规划操作序列，与操作系统互动（包括执行 Shell 命令、点击 GUI 等），最后验证执行结果。这对本文 Shell 命令生成系统的设计具有直接的参考价值。

AgentBench^[7] 提供了一个系统化的 Agent 评估框架，通过 3600+ 个任务评估 LLM 在不同 Agent 场景中的性能。这样的评估基准对于推动 Agent 系统的进步至关重要。

自主 Agent 的系统化综述^[7] 总结了 LLM Agent 在多个领域的应用进展。

2.7 模型参数高效训练与量化

2.7.1 大模型量化的必要性

大语言模型的参数量通常以十亿（Billion）为单位，以 FP32 精度存储时，7B 参数模型约需 28 GB 内存，超出消费级 GPU（通常 4–16 GB VRAM）的承载能力。量化（Quantization）技术通过降低参数的数值精度将模型压缩至可在消费级硬件上运行的规模，是大模型本地化部署的核心使能技术。

量化方法可按压缩时机分为两类：训练后量化（Post-Training Quantization, PTQ）在训练完成后直接对权重进行精度转换，无需重新训练，适合快速部署；量化感知训练（Quantization-Aware Training, QAT）在训练阶段引入量化误差反馈，精度损失更小但需要额外的计算资源。本系统采用 PTQ 方案，基于 Hugging Face Transformers 集成的 BitsAndBytes 库实现。

2.7.2 NF4 量化原理

BitsAndBytes 库提供的 4-bit 量化方案支持两种数值格式：FP4（Float 4-bit）和 NF4（Normal Float 4-bit）。NF4 由 Dettmers 等人在 QLoRA 工作中提出^[7]，其核心思想是：LLM 的权重在经过适当归一化后近似服从零均值正态分布，因此可以设计一种信息论意义上近优的 4-bit 数字表示，使量化级别在正态分布高概率密度区域分布更密集，从而降低量化误差的期望值。相比 FP4 的均匀量化，NF4 在同等比特宽度下能够以更少的量化误差表示正态分布的权重，是当前 4-bit 量化的主流选择。

2.7.3 双重量化与本系统配置

为进一步压缩内存开销，本系统同时启用双重量化（Double Quantization）：对存储量化常数（scaling factors）所用的 FP32 精度本身再次进行 8-bit 量化，平均每个参数额外节省约 0.37 bit 的存储空间。

Hugging Face BitsAndBytes 库将 NF4 与双重量化集成为开箱即用的接口，支持在加载模型时通过统一配置同时启用 4-bit 量化、双重量化与计算精度设置，矩阵乘法阶段通常以 BF16 或 FP16 精度执行以兼顾速度与数值稳定性。Transformers 的 `device_map` 机制可将模型权重在 GPU 与 CPU 内存之间自动均衡分配，使超出单卡 VRAM 容量的模型得以在消费级硬件上完成推理。本文采用上述技术组合部署大语言模型，具体参数配置详见第三章系统设计。

2.8 本章小结

本章介绍了与本系统相关的核心技术背景。首先阐述了检索增强生成（RAG）的基本原理、系统组成与技术发展脉络；其次介绍了稠密向量检索的数学基础、中文嵌入模型的选型背景及文档分块策略；然后分析了稀疏检索与向量检索的互补性，并介绍了基于加权乘积的混合检索重排方法；再从风险特征与防护层次两个角度概述了 Shell 命令安全的技术背景；最后介绍了大模型量化技术的基本原理与 NF4、双重量化方案的核心思路。这些技术构成了本系统设计与实现的理论基础，后续章节将在此基础上展开具体的系统设计与实验分析。

第三章 系统架构

3.1 整体设计

3.1.1 设计目标

本系统的设计目标为：以自然语言作为输入，生成对应的 Shell 命令并辅助执行，同时保障命令执行的安全性与多轮会话的上下文连贯性。具体要求包括：支持中文口语化表达；适配 Bash 与 Zsh 两种主流 Shell；集成领域知识检索以减少命令幻觉；提供三级安全执行保障。

3.1.2 总体架构

系统采用客户端-服务端分离架构，如图 3-1 所示。两个进程通过 HTTP 接口通信，职责明确分离。

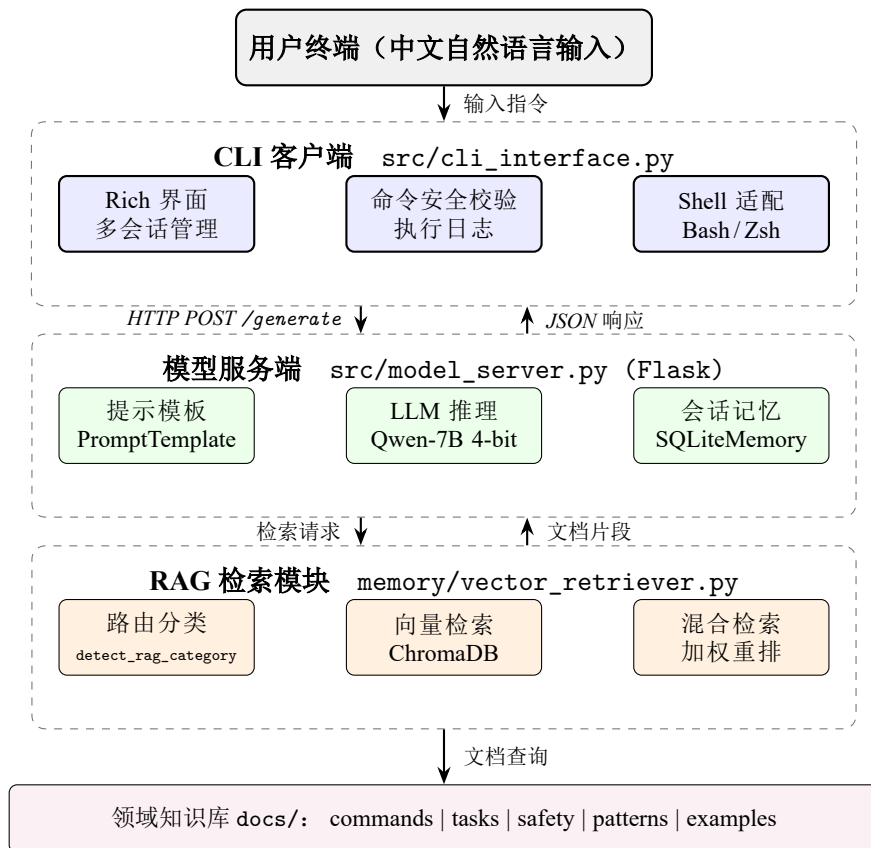


图 3-1 系统总体架构图

采用该架构的主要设计动因包括：

1. 资源特性解耦：LLM 推理属于计算密集型任务，对 GPU 显存与内存带宽要

求显著高于 CLI 交互逻辑；将两者拆分可避免终端交互流程被重计算路径阻塞。

2. 部署与扩展灵活性：服务端可独立部署在具备 GPU 资源的节点，客户端仍保持轻量；在同一服务端上可并发服务多个前端实例，提升复用性。
 3. 接口稳定性：以 HTTP 作为边界后，模型实现可在服务端内部替换（如更换模型版本、量化策略或推理后端）而不影响客户端调用方式。
 4. 工程可维护性：模块边界清晰后，CLI 交互逻辑与模型推理逻辑可分别测试、迭代与发布，降低系统耦合度。
- 模型服务端：负责 LLM 推理、RAG 检索和会话记忆管理，以 Flask 框架对外暴露/generate 等 HTTP 接口。
 - CLI 客户端：负责用户交互、命令安全校验与执行日志，通过 QwenHTTP 类与服务端通信，以 Rich 库渲染终端界面。

系统各核心模块的对应关系见表 3-1。

表 3-1 系统核心模块

模块	源文件	核心职责
CLI 交互模块	src/cli_interface.py	用户输入、命令展示、执行确认
模型服务模块	src/model_server.py	LLM 推理、上下文组装、会话管理
RAG 检索模块	memory/vector_retriever.py	向量索引、混合检索、重排序
路由分类模块	src/rag_routing.py	查询到知识库类别的映射
记忆管理模块	memory/sqlite_memory.py	分层对话记忆的存储与检索
命令安全模块	src/command_safety.py	权限分级、预执行校验、效果模拟
执行日志模块	src/execution_logger.py	JSONL 日志记录与 CSV 导出
Shell 适配模块	config/shell_config.py	Bash/Zsh 语法差异适配

3.2 CLI 客户端模块

CLI 客户端由 ShellAgentCLI 类实现，承担以下职责：

用户交互：基于 Rich 库提供彩色面板、语法高亮和交互式确认提示，支持多会话管理（new/chats/use 等内置命令）。每个会话拥有独立的 session_id，对应服务端独立的记忆上下文。

响应解析：服务端返回 JSON 格式字符串，包含 command、explanation、warning 三个字段。由于 LLM 输出具有概率性与格式漂移特征，即使通过提示模板约束 JSON 格式，仍可能出现引号缺失、转义错误或字段不完整等问题。因此系统采用多级容错解析策略，以提高命令提取的鲁棒性与可用性。客户端实现了四

级容错解析策略：

1. 标准 JSON 解析；
2. 修复常见转义错误后再次解析；
3. 基于正则表达式的字段提取回退；
4. 基于标签格式（命令:/说明:/警告:）的字段回退。

采用该多级策略的原因在于：LLM 输出具有概率性与格式漂移特征，即使通过提示工程约束 JSON 格式，仍可能出现引号缺失、转义错误或字段不完整等问题。四级容错机制通过“严格解析优先、局部修复兜底、字段回退保底、标签格式兜底”的递进设计，在不显著增加解析复杂度的前提下提升了命令提取鲁棒性，降低了因输出格式异常导致的交互中断概率。

Shell 适配：通过 `config/shell_config.py` 中的 `ShellSyntax` 数据类维护 Bash 与 Zsh 的语法差异，涵盖数组定义格式、变量引用方式、间接变量展开（`${!name}` vs `${(P)name}`）及数组索引基准（Bash 从 0 起，Zsh 从 1 起）等关键差异，在命令生成提示中注入目标 Shell 信息以引导模型输出对应语法。

多会话管理：CLI 客户端支持多个独立的对话会话，允许用户在同一进程中管理多个任务上下文。每个会话拥有唯一的 `session_id`（在服务端对应独立的记忆上下文），用户可通过内置命令进行会话操作：

- `/new [标题]`：创建新会话。若不指定标题，自动编号为“对话 N”。
- `/chats`：列出所有会话，标记当前活跃会话。
- `/use < 选择器 >`：切换到指定会话，选择器支持序号（如 1）、标题或 `session_id`。
- `/delete < 选择器 >`：删除指定会话（至少保留一个会话）。

会话状态通过 JSON 格式持久化存储在 `data/cli_chats.json`，包含活跃会话索引、会话计数器及会话元数据列表。应用启动时自动加载已保存的会话列表；若无会话记录则自动创建首个会话“对话 1”。该机制使用户能够在多个独立的工作流程中切换，例如同时进行日志分析与系统配置两个任务，每个会话维持独立的对话历史与上下文，避免不同任务的记忆混淆。

每当会话切换时，客户端会重新初始化当前会话的工作记忆（见下文）。各会话的完整对话记忆由服务端 `SQLiteMemory` 基于 `session_id` 独立管理，两个会话间不共享对话历史、命令执行记录或上下文摘要，从而实现了不同任务间的隔离。

工作记忆：客户端维护一个轻量级工作记忆（`_working_memory`），记录上一条命令（`last_command`）、上一次输出（`last_output`）及实体列表（`items`），用

于支持“将这些文件移动到…”此类依赖前序结果的后续指令解析。

分步任务执行：系统能够自动识别包含多个步骤的复杂任务请求，并以循环方式逐步生成与执行命令。启发式判定逻辑通过检测用户输入中的顺序连接词（“然后”、“再”、“最后”、“接着”等）与复杂任务关键词（“部署”、“迁移”、“安装”、“配置”等）判断是否触发分步执行模式。需要指出的是，该启发式规则对隐式多步意图（如未使用上述连接词但实际包含多阶段操作的请求）存在漏判风险；漏判时系统仍会生成完整命令链，但以单次非分步模式呈现。

分步执行循环最多支持 12 步，其中包含以下四条中断路径：

1. JSON 格式损坏自动降级退出：每步的模型输出均经过四级容错解析（标准解析 → 转义修复重解析 → 正则字段提取 → 标签格式回退）。若全部降级策略均未能提取到有效命令字段（`command=""`），系统将该步视为任务完成信号并立即退出循环，避免将格式异常传播到后续步骤。
2. 安全校验失败强制中断：每步在执行前均经过语法校验、风险分级、效果模拟三步预执行流水线（详见第五章 章）。若命令被判定为 `blocked` 级别或高风险命令且用户拒绝确认，分步循环立即终止，后续步骤不再生成。
3. 用户逐步确认中断：每步命令均需用户显式确认（“执行该步骤命令？”）后方可执行；执行完毕后还需用户确认是否继续（“继续下一步？”）。任一确认环节拒绝均可即时终止循环。
4. 步骤上限保护：循环至多执行 12 步后自动暂停，防止因模型持续生成命令而造成失控执行。

关于命令执行失败（非零返回码）的处理：当某步命令执行后 Shell 返回非零退出码时，`subprocess.run` 捕获的 `stderr` 输出将随 `stdout` 一并写入工作记忆的 `last_output` 字段，作为后续步骤提示的上下文供模型参考。当前系统不主动对非零退出码执行自动中断，而是由用户在“继续下一步？”确认时根据屏幕上已打印的错误输出手动决定是否终止，保留了用户对异常情况的最终控制权。该机制使系统能够处理“先压缩日志目录，然后将压缩包移至备份服务器，最后清理原目录”等涉及多条命令的连续操作任务。如图 3-2 所示，系统以统一的 CLI 界面组织用户输入、命令展示、说明文本与执行反馈，形成从提问到结果回显的一体化交互流程。

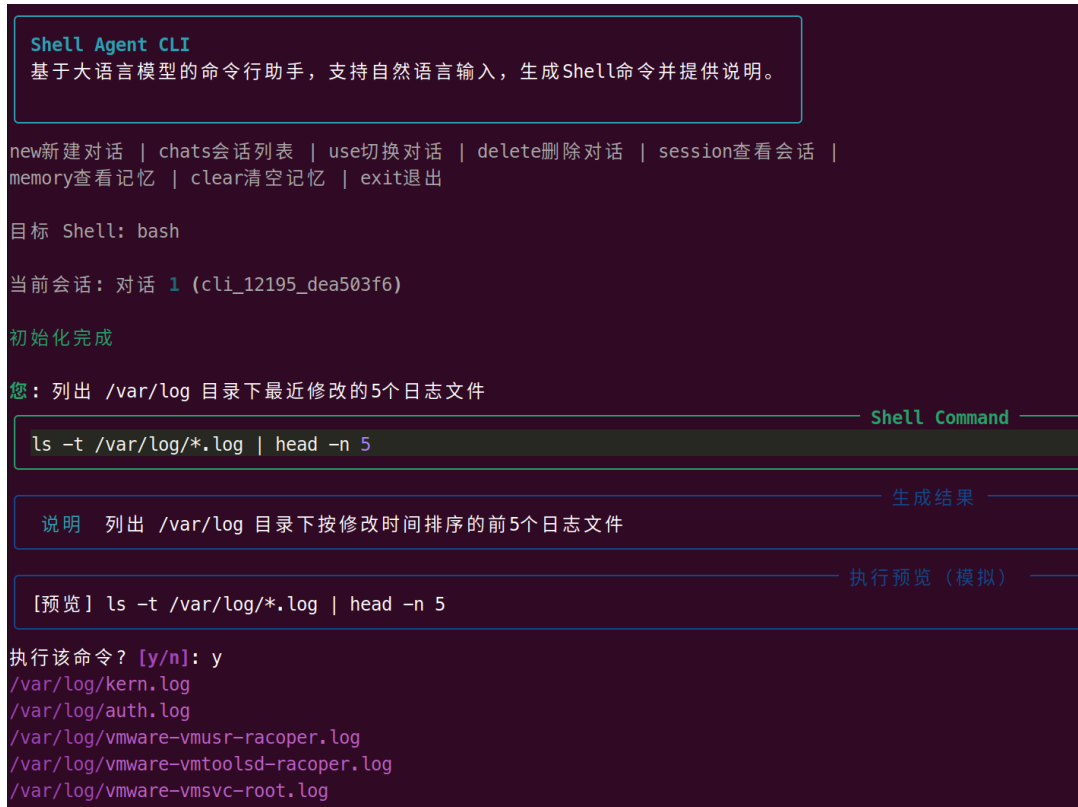


图 3-2 系统 CLI 整体运行界面实拍

3.3 模型服务器模块

3.3.1 LLM 配置

服务端以本地量化 Qwen-7B 模型为推理核心，采用 BitsAndBytes 库进行 4-bit NF4 量化^[2]（嵌套量化），通过 `device_map="balanced"` 实现 GPU/CPU 混合部署，显存上限设置为 7GB，超出部分卸载至 CPU 内存（上限 32GB）。量化配置如下：

- 量化类型：NF4（Normal Float 4-bit）
- 嵌套量化：启用（双重量化压缩量化常数）
- 计算精度：float16
- CPU 卸载：启用

模型在服务启动时仅加载一次（`model.eval()`），后续所有请求共享同一模型实例，避免重复加载开销。

3.3.2 提示模板

服务端使用固定的结构化提示模板，通过 LangChain 的 `PromptTemplate` 组装，包含以下占位符：

- `{summary}`：当前会话的摘要记忆；

- `{recent_history}`: 最近若干轮对话记录;
- `{relevant_memory}`: RAG 检索到的相关文档片段;
- `{input}`: 当前用户输入;
- `{target_shell}`: 目标 Shell 类型 (`bash` 或 `zsh`)。

模板要求模型始终以 JSON 格式返回, 包含 `command`、`explanation`、`warning` 三个字段, 并给出角色行为规则 (闲聊时不生成命令、危险操作提示警告等)。

3.3.3 会话管理

服务端为每个 `session_id` 维护独立的 `SQLiteMemory` 实例, 存储于共享的对话记忆数据库。为防止内存无限增长, 实现了基于 TTL (Time-To-Live) 的会话驱逐机制: 空闲超过 `SHELL_AGENT_SESSION_TTL` 秒 (默认 3600 秒) 的会话将被自动清除。

3.4 RAG 检索模块

RAG 检索模块以环境变量 `SHELL_AGENT_ENABLE_RAG` (默认开启) 控制启用状态, 服务端启动时初始化 `VectorRetriever` 实例, 若 `SHELL_AGENT_RAG_DOCS` 指定了文档路径则同步构建/更新索引。

检索流程如下:

1. 由路由分类模块将用户查询映射至五类知识库之一 (或不过滤);
2. 以类别过滤条件执行混合检索;
3. 若类别过滤结果为空, 则回退至全库无过滤检索 (保底策略);
4. 将检索到的文档以 `<doc>` 标签包裹后注入提示的 `relevant_memory` 字段。

3.5 内存管理模块

分层记忆系统由 `HierarchicalMemory` 类实现, 包含三个层次:

1. 短期记忆 (Short-Term Memory): 保留最近 N 轮 (默认 10 轮) 完整对话记录, 每轮包含角色 (`user/assistant`)、内容、轮次编号与关键词索引。
2. 长期记忆 (Long-Term Memory): 基于关键词倒排索引, 从历史对话中检索与当前输入语义相关的片段, 弥补短期记忆窗口限制。关键词索引在初始化时从数据库重建, 保证持久化后重启可用。
3. 摘要记忆 (Summary Memory): 对话摘要以字符串形式持久化于 `session_summary` 表, 在提示中作为全局上下文背景, 压缩长期会话信息。

所有数据库操作通过 `threading.Lock` 加锁, 保证并发安全。

3.6 系统端到端处理流程

为直观呈现各模块协作关系，本节以一次典型用户交互为例，描述请求从 CLI 输入到命令输出的完整处理流程。

第一步——用户输入与请求发送。用户在 CLI 终端输入自然语言需求（如“帮我查找当前目录下超过 100MB 的文件”）。ShellAgentCLI 将输入文本与当前会话的 `session_id` 打包为 JSON，通过 HTTP POST 请求发送至模型服务端的 `/generate` 接口。

第二步——RAG 检索。服务端接收请求后，首先对用户输入进行关键词路由分类，确定目标知识库类别（本例映射至 `commands` 类）。随后以类别过滤条件执行向量检索，对用户输入进行向量编码，在 ChromaDB 中检索语义相近的文档块，并结合关键词命中率进行混合重排，返回 Top-5 文档块。若过滤后结果为空则回退至全库检索。

第三步——提示组装与 LLM 推理。服务端从 HierarchicalMemory 中读取当前会话的短期记忆、长期记忆关键词匹配片段及摘要记忆，连同 RAG 检索结果一并填入提示模板，构造完整提示文本。提示文本提交至本地量化 Qwen-7B 模型进行自回归推理，生成包含 `command`、`explanation`、`warning` 字段的 JSON 字符串。

第四步——响应解析与安全校验。CLI 客户端对返回的 JSON 字符串执行四级容错解析。提取出命令字符串后，依次执行预执行三步流水线：语法校验（检查括号平衡、引号配对）、风险分级（通过七级规则链确定 `risk_level`）、效果模拟（对文件操作类命令预览影响范围）。

第五步——用户确认与执行记录。CLI 以 Rich 面板展示命令、解释与风险提示，对中危及以上级别命令要求用户键入确认字符方可执行。执行完成后，ExecutionLogger 将本次交互的完整信息写入 JSONL 日志，同时 HierarchicalMemory 更新会话短期记忆，支持后续多轮上下文关联。

上述五步流程覆盖了从自然语言输入到安全命令执行的全链路，各模块通过接口边界清晰分工，任一模块均可独立测试或替换。

3.7 本章小结

本章介绍了系统的总体架构与各核心模块的设计。系统采用客户端-服务端分离架构，模型服务端负责 LLM 推理、RAG 检索与会话记忆管理，CLI 客户端负责用户交互、命令安全校验与执行日志，两端通过 HTTP 接口解耦。各模块职责清晰，具备良好的可测试性与可扩展性。第四章将详细介绍领域 RAG 检索系统的具体实现，第五章将介绍命令安全执行框架的设计细节。

第四章 领域 RAG 检索系统

4.1 知识库构建

4.1.1 文档分类体系

知识库位于 docs/ 目录，按内容性质划分为五个类别，每类对应一个子目录，如表 4-1 所示。

表 4-1 知识库类别体系

类别	目录	文档数	内容描述
命令参考	commands/	21	常用 Shell 命令用法卡片
安全策略	safety/	9	高危操作防范与权限策略
操作流程	tasks/	12	多步骤 SOP 操作指南
模式文档	patterns/	2	Shell 兼容性与安全执行模式
示例文档	examples/	1	典型使用场景示例

知识库共 45 篇文档（未含模板），按类别分布如下：命令参考 21 篇、任务 SOP12 篇、安全策略 9 篇、设计模式 2 篇、典型示例 1 篇，覆盖约 150 种用户意图和操作场景。命令类别文档（如 find_large_files.md、change_owner_chown.md）以结构化 Markdown 格式编写，包含适用场景、推荐命令、参数解释、风险说明、回滚方案等固定章节；任务类文档（如 backup_directory_sop.md、log_cleanup_sop.md）则以多步骤操作指南形式撰写，每步包括操作目标、命令示例、风险说明和验收标准。

4.1.2 文档增强策略

通用 LLM 生成的自然语言查询与知识库文档的书面表述之间存在词汇鸿沟（Vocabulary Gap）问题：用户可能以口语化、中英混杂或隐晦描述提出需求，而文档以规范技术语言撰写，导致语义相关但表面形式差距较大的查询难以命中正确文档。

为系统性缓解此问题，对知识库中所有文档统一添加以下增强结构：

1. 同义问法（Synonym Queries）：为每篇文档预设 10 至 20 条覆盖口语化、被动语态、中英混杂、简略表达等多样化形式的同义问法，直接嵌入文档文本。由于嵌入模型在块级别建立索引，同义问法与文档正文共同构成块内容，查询的嵌入向量更容易与对应块的向量接近。例如，find_large_files.md 中

包含：“帮我找出当前目录超过 100MB 的文件”、“如何查找大文件？”等问法。

2. 覆盖失败查询 (Failure Query Coverage): 将开发集 (Dev Set) 中历史检索失败的原始查询 (即 RAG 无法命中正确文档的查询) 直接嵌入对应文档的“覆盖失败 Query”章节。本文明确禁止将最终测试集 (80 条自然语言查询) 中的查询写回知识库, 以避免数据泄露 (Data Leakage)。此策略将已知失败案例显式编码为文档内容, 确保相同或相似查询在下次检索时能够命中正确文档。
3. 文档元信息 (Document Metadata): 每篇文档标注类别标签、典型意图描述、风险等级和适用 Shell 类型, 用于支持精确的元数据过滤检索。

文档增强策略将基于文档模板的查询集 MRR 从约 0.92 提升至 1.0 (满分), 表明同义问法注入能够提升查询与文档块之间的向量相似性, 在标准化查询条件下改善了检索匹配效果。

4.1.3 索引构建

索引构建由 RAG 索引构建模块执行, 或在服务端启动时由 SHELL_AGENT_RAG_DOCS 环境变量触发自动构建。

构建流程如下:

1. 递归扫描指定路径下的 .md、.txt 等文件;
2. 对每个文件执行 Markdown 感知分块 (参数: 块大小 600 字符, 重叠 100 字符);
3. 对每个块调用嵌入模型生成向量;
4. 将 (向量, 文本, 元数据) 三元组写入 ChromaDB 集合。

对于已索引的文档, 系统基于内容哈希检测变更, 仅对新增或变更的块执行增量更新, 避免重复索引。

4.1.4 评估驱动的迭代扩库方法论

知识库的质量直接决定 RAG 系统的检索上限。本系统采用“评估-分析-扩充”的迭代扩库方法论, 以量化指标驱动知识库的持续改善。

在数据治理上, 本文将评估查询分为开发集与测试集两部分: 开发集用于失败分析与知识库扩充; 测试集用于最终一次性评估。两者严格互斥, 且测试集在最终评估前冻结, 不参与任何同义问法补充、失败查询回填或文档改写。

迭代流程如下: 第一步——基准评估。运行检索评估模块对当前知识库执行全量评估, 记录每条查询的命中情况。第二步——失败分析。统计未命中查询

(SrcHit=0)，人工分析失败原因：若目标文档缺失，则新增对应知识文档；若目标文档存在但未被召回，则将开发集失败查询追加至该文档的“覆盖失败 Query”字段，强制建立查询与文档之间的语义关联。第三步——重建索引。更新文档后重新运行索引构建脚本，并再次执行评估，确认各指标提升后进入下一轮迭代。

通过八轮迭代和持续改进，知识库规模从最初约 53 个文本块扩充至约 88 个文本块，来源命中率 (SrcHit) 在开发集模板查询上从 0.880 提升至 1.000，在自然语言查询集上从无基准数据提升至 0.950，验证了评估驱动扩库方法论在小规模领域知识库构建中的实际效果。

最新一轮 (Round 8，知识库重建) 在开发集模板查询 (100 条) 上的结果为：A 臂 (仅向量检索) SrcHit 达到 1.0，MRR 达到 1.0，nDCG 为 0.9785，Recall@1 为 0.98，表明当前知识库对开发阶段模板查询在评估集上实现完全覆盖。该结果仅用于开发诊断上界，不作为最终泛化性能结论。

表 4-2 知识库迭代历程中 A 臂检索指标演进 (开发集模板查询，100 条，仅向量检索)

迭代轮次	主要操作	SrcHit	MRR	nDCG	Recall@1
Round 1	初始知识库 (≈ 53 chunks)	0.880	0.895	0.880	0.750
Round 5	新增任务和安全类核心文档	0.970	0.983	0.952	0.850
Round 7	全量扩充至 88 chunks	1.000	1.000	0.982	0.980
Round 8	知识库重建验证	1.000	1.000	0.9785	0.980

从表中可以观察到的关键规律：Round 1 至 Round 4 阶段，SrcHit 停滞在 0.86–0.88 区间，说明仅通过同义问法与元数据标注的调整，无法突破目标文档缺失导致的召回上限。Round 5 起新增了任务类与安全类的 12+9 篇核心文档，SrcHit 跳升至 0.97，MRR 从 0.925 提升至 0.983，实验结果表明知识库覆盖范围的扩展与检索性能提升存在明显相关性。Round 7 与 Round 8 确认了在 88 个文本块 (45 篇原始文档) 的规模下，对模板查询集在 88 个文本块规模下实现完全覆盖 (SrcHit=1.0, MRR=1.0)，性能指标达到理论上限。每轮迭代的原始数据均保存至实验报告目录，为第六章实验分析提供了详细的历史对比依据。需要强调：表 4-2 反映的是开发集上的迭代优化轨迹，并不等价于最终测试集上的泛化能力。

4.2 嵌入与索引

4.2.1 中文嵌入模型选型

本系统选用 BAAI/bge-small-zh-v1.5 作为文本嵌入模型^[9]。该模型由北京人工智能研究院（BAAI）发布，专为中文语义理解优化，在多个中文语义相似度基准测试中表现优异。模型参数量约为 33M，推理速度快，适合本地部署场景，向量维度为 512 维，经 L2 归一化后存储为单位向量，便于余弦相似度计算。

4.2.2 分块策略与参数

本系统采用 Markdown 感知自适应分块策略，核心参数见表 4-3。

表 4-3 文档分块参数配置

参数	默认值	说明
chunk_size	600 字符	单块最大字符数
chunk_overlap	100 字符	相邻块重叠字符数
max_chunks_per_source	2	单文档最多返回的块数（检索时）
distance_threshold	1.2	余弦距离过滤阈值

分块时优先按 Markdown 标题（##/###）和双换行符切分，保留文档语义结构；当段落超过 chunk_size 时，回退至字符级滑动窗口切分（步长 = chunk_size - chunk_overlap）。

4.3 混合检索与重排策略

4.3.1 向量检索候选集构建

对用户查询 q ，系统首先通过嵌入模型生成查询向量 q ，通过 ChromaDB 接口获取余弦距离最小的 $3k$ 个候选块（其中 k 为最终返回数量，默认 $k = 4$ ）。每个候选块的向量相似度分数由余弦距离 $dist$ 转换为：

$$s_{\text{vec}}(d) = \frac{1}{1 + dist} \quad (4-1)$$

若指定了类别过滤条件，则在 where 参数中添加元数据过滤约束，使向量检索仅在对应类别的块中进行。

4.3.2 关键词候选集构建

关键词检索通过扫描集合中的全部文档块（上限 `keyword_scan_limit`，默认 2000 块）计算每个块的关键词命中率。查询首先被分词为词元列表（中文保留完整短语，英文按最小长度过滤），对每个块计算：

$$s_{kw}(d) = \min \left(1.0, \frac{\text{命中词元数}}{|\text{查询词元集}|} + 0.25 \cdot 1[\text{完整短语命中}] \right) \quad (4-2)$$

关键词候选集按 s_{kw} 降序排列，取前 $3k$ 条。

4.3.3 合并与重排

向量候选集与关键词候选集以块 ID 为键合并：已在向量候选中出现的块，将其关键词分数填入；仅出现在关键词候选中的块，当且仅当 $s_{kw}(d) \geq 0.5$ 时方可纳入合并集合，防止低质量关键词候选污染结果。

合并后按混合分数降序排列并截取 Top- k ：

$$\text{score}(d) = s_{vec}(d) \times (1.0 + 0.2 \cdot s_{kw}(d)) \quad (4-3)$$

关键词权重系数 $\alpha = 0.2$ 经在评估集上调参确定，既能体现关键词补强效果，又不至于过度干扰向量相关性排序。

4.4 关键词路由过滤

4.4.1 路由机制设计

关键词路由将用户查询映射至五类知识库之一，在检索时施加类别过滤以缩小候选集，从而提升检索精度与效率。

路由规则按优先级顺序执行，一旦命中立即返回，不继续匹配后续规则，具体执行流程由规则优先级与命中即返回策略共同决定。

4.4.2 路由规则

路由规则的优先级设计如表 4-4 所示。

优先级 1（精确短语预检）专门处理包含安全相关词汇但实为任务类的查询，例如“安全导出日志”若按安全关键词路由至 `safety` 类别将导致错误，因此在规则 3 之前提前捕获。同理，优先级 2 的 `tar.gz` 规则确保压缩相关查询不被任务类关键

表 4-4 关键词路由规则（按优先级排列）

优先级	规则描述	触发关键词示例	目标类别
1	精确任务短语预检	“安全导出”、“导出日志”	tasks
2	tar.gz 压缩短语	“tar.gz”、“tar ...gz”	commands
3	安全策略关键词	“安全”、“危险”、“黑名单”、“dry-run”	safety
4	任务流程关键词	“备份”、“巡检”、“定时任务”、“SOP”	tasks
5	Shell 模式关键词	“bash”、“zsh”、“差异”、“预览”	patterns
6	示例类关键词	“例子”、“示例”、“example”	examples
7	默认	（不匹配任何规则）	commands

词（如“备份”）错误路由。

4.4.3 路由的保底策略

服务端在检索时，若路由过滤后的候选集为空（即当前类别无相关文档），自动回退至全库无过滤检索，确保系统在知识库覆盖不足时仍能返回最佳结果而非空响应。

路由规则通过 22 个单元测试用例验证（含跨类边界案例），全部通过，保证了规则的正确性。

4.5 本章小结

本章详细介绍了领域 RAG 检索系统的完整设计与实现。在知识库构建方面，通过同义问法注入、失败查询覆盖和文档元信息标注三项文档增强策略，在一定程度上缓解了词汇鸿沟问题。在检索策略方面，提出了基于加权乘积的混合检索重排方法，并设计了带优先级的关键词路由过滤机制。第六章将通过消融实验对各模块的独立贡献进行系统性评估。

第五章 命令安全执行系统

5.1 设计目标

Shell 命令执行具有强副作用性，LLM 生成的命令可能因幻觉或提示注入^[?] 导致危险操作。本系统的安全执行框架遵循默认拒绝（Default Deny）原则，从权限控制、预执行校验和执行审计三个层次构建防护，设计目标如下：

1. 阻止已知高危命令（如 `rm -rf /`、`mkfs`、`dd of=/dev/`等）的执行；
2. 对具有写操作风险的命令要求用户二次确认；
3. 在实际执行前以只读模拟方式预览命令效果；
4. 记录所有命令的执行历史，支持事后审计。

5.2 三级权限分级机制

5.2.1 风险等级定义

命令安全模块的核心数据结构为 `SafetyResult`，包含四个字段：是否允许执行（`allowed`）、风险等级（`risk_level`）、影响范围描述（`scope_description`）和拒绝原因（`reason`）。

风险等级分为四级，如表 5-1 所示。

表 5-1 命令风险等级定义

等级	标识	是否允许	说明
低风险	low	允许直接执行	只读查询类命令，无副作用
中风险	medium	允许，显示提示	具有写操作但影响范围有限
高风险	high	允许，需二次确认	批量删除、递归修改权限等
拦截	blocked	拒绝执行	命中黑名单或超出白名单

5.2.2 分级判定规则

分级判定规则按以下优先级顺序执行，一旦命中立即返回，不继续匹配后续规则：

第一级——字符串黑名单（`blocked`）：精确子串匹配，命令文本中包含预定义危险字符串即拒绝执行。典型条目包括：`mkfs`（格式化磁盘）、`dd of=/dev/`（磁盘直写）、`:(){ :|:& };`（fork 炸弹）、`shutdown`、`sudo su` 等权限提升命令，以及覆盖系统关键文件的重定向操作（`> /etc/passwd` 等）。

第二级——正则黑名单 (**blocked**): 正则表达式匹配, 用于捕获参数组合复杂的危险模式, 包括: `rm -rf /` (根目录递归删除)、`rm ~` (删除家目录)、`rm -*` (通配符删除) 及 `--no-preserve-root` 等危险标志。

第三级——受保护路径写操作 (**blocked**): 当命令的动词为写类操作 (`rm`、`mv`、`chmod` 等), 且目标路径属于受保护路径集合 (`/etc`、`/boot`、`/bin`、`/sbin`、`/lib` 等) 时, 拒绝执行。

第四级——高危模式 (**high**): 正则匹配高危但不绝对禁止的模式, 包括: `rm -r*` (递归删除非根目录)、`find ... -delete` (批量删除)、`chmod -R` (递归改权限)、`curl|bash` (管道执行远程脚本)、`sudo` (权限提升, 允许但要求二次确认) 等。

第五级——白名单前缀 (**low**): 命令前缀匹配白名单, 包含 `ls`、`cat`、`grep`、`find` (不带 `-delete`)、`git status/log/diff` 等纯查询类命令, 直接放行。

第六级——中危模式 (**medium**): 正则匹配有写副作用但非高危的命令, 包括: `rm` (不带 `-r` 且无路径穿越)、`mv`、`cp`、`chmod` (非递归)、`systemctl restart/stop`、`pip/apt` 等软件包管理命令。

第七级——默认拒绝 (**blocked**): 未被任何白名单规则命中的命令, 默认归为 **blocked**, 提示用户联系管理员申请执行权限。

5.2.3 典型命令分类样例

为直观说明七级规则链的覆盖范围, 表 5-2 列举了 15 条具有代表性的命令及其分类结果, 所有判定结果均已通过专项测试用例验证。

值得注意的是最后一条 `vim /etc/nginx.conf`: `vim` 不在任何白名单前缀中, 亦未命中高危或中危正则模式, 由于系统采用默认拒绝策略, 交互式编辑器命令被归为 **blocked**。这体现了“宁可误拦截, 不可漏拦截”的设计原则——用户若需编辑系统配置文件, 应在独立终端中执行, 而非通过 Shell 智能体代理执行。

5.2.4 规则链的边界情况与设计权衡

七级规则链在设计时面临误拦截 (**False Positive**) 与漏拦截 (**False Negative**) 的权衡。为遵循“最小权限、默认拒绝”原则, 系统有意将误拦截率设置得较高 (即宁可误拦截合法命令, 也不放行危险命令), 第七级默认拒绝规则正是这一设计意图的体现。

在工程实践中, 以下几类边界情况值得特别关注:

管道与命令替换组合。形如 `cat file | grep keyword | xargs rm -f` 的组合命令, 虽然各子命令单独看属于不同风险级别, 但整体语义已构成批量删除。规

表 5-2 典型命令安全分级样例

命令示例	命中规则级别	判定结果
ls -la /home	第五级·白名单前缀 (ls)	low, 直接执行
cat /etc/hosts	第五级·白名单前缀 (cat)	low, 直接执行
grep -r "error" /var/log	第五级·白名单前缀 (grep)	low, 直接执行
git status	第五级·白名单前缀 (git status)	low, 直接执行
cp src.txt dst.txt	第六级·中危模式 (^cp\b)	medium, 提示后执行
chmod 755 script.sh	第六级·中危模式 (^chmod\b)	medium, 提示后执行
pip install requests	第六级·中危模式 (^pip\b)	medium, 提示后执行
rm -rf /tmp/test	第四级·高危模式 (rm -r*)	high, 需二次确认
chmod -R 755 /var/www	第四级·高危模式 (chmod -R)	high, 需二次确认
sudo systemctl restart nginx	第四级·高危模式 (^sudo)	high, 需二次确认
curl http://x.com/s.sh bash	第四级·高危模式 (curl bash)	high, 需二次确认
rm -rf /	第二级·正则黑名单	blocked, 拒绝执行
dd of=/dev/sda if=/dev/zero	第一级·精确黑名单 (dd of=/dev/)	blocked, 拒绝执行
mkfs.ext4 /dev/sdb1	第一级·精确黑名单 (mkfs)	blocked, 拒绝执行
vim /etc/nginx.conf	第七级·默认拒绝	blocked, 拒绝执行

则链通过整体字符串匹配（而非分词解析子命令）检测此类模式，例如第二级正则黑名单包含 `xargs rm` 模式，确保此类组合被拦截。

变量展开的延迟特性。形如 `rm -rf $DIR` 的命令，在静态分析阶段无法确定 `$DIR` 的实际值，若其在运行时展开为 `/` 则后果严重。对此，规则链将包含变量展开的删除命令（`rm.*${?[A-Z]...}`）纳入高危模式，要求用户二次确认方可执行。

交互式命令的阻塞风险。`top`、`vim` 等交互式命令若被直接通过 `subprocess` 执行，会导致 CLI 进程挂起。白名单规则将此类命令排除在自动执行范围之外，标记为 `blocked` 并提示用户在独立终端中运行。

上述边界情况的处理逻辑均通过 `test_command_safety.py` 中的专项测试用例覆盖，保证了规则链在复杂命令场景下的正确性。

5.3 预执行校验流水线

5.3.1 三步流水线设计

每次用户确认执行命令前，CLI 客户端触发预执行校验流水线，依次执行三个校验步骤，如图 5-1 所示。

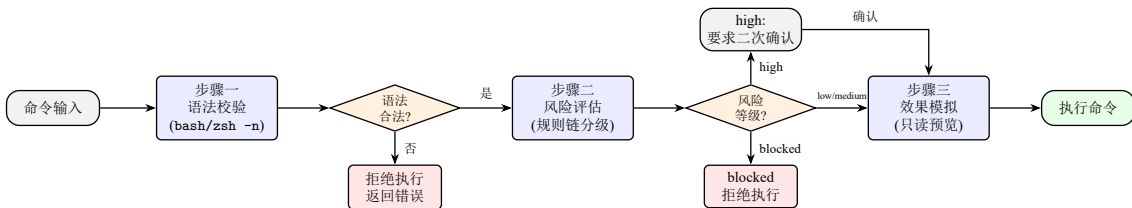


图 5-1 预执行三步校验流水线

步骤一——语法校验：调用目标 Shell 的 `-n` 标志对命令进行语法检查（不实际执行），优先使用系统中可用的 `bash/zsh` 二进制执行校验。当 Shell 二进制不可用或超时（5 秒限制）时，回退至内置的轻量级语法检查器，该检查器可检测未闭合的命令替换 `$(...)`、未闭合引号及 `for ... in` 语句缺少迭代列表等常见错误。语法校验失败时直接拒绝执行，不进入后续步骤。

步骤二——风险评估：按第 5.2.2 节所述的七级判定规则对命令进行分级。风险等级为 `blocked` 时直接拒绝；等级为 `high` 时向用户展示影响范围描述并要求二次确认；等级为 `medium` 时展示提示信息后继续。

步骤三——效果模拟：对通过前两步校验的命令，以只读方式生成执行效果预览，用户可在确认预览结果后再决定是否执行。具体模拟策略如表 5-3 所示。

如图 5-2 所示，当系统检测到危险命令或高风险操作时，CLI 界面会给出拦截提示或风险警告，并要求用户进一步确认。

表 5-3 效果模拟策略

命令类型	模拟方式	说明
rm [args] <target>	find <target> -maxdepth 3	列出将被删除的文件
find ... -delete	去掉-delete 后执行	列出将被删除的匹配文件
chmod/chown <perm> <target>	stat <target>	显示目标当前权限/属主
mv <src> <dst>	文本提示	说明源与目标路径
cp <src> <dst>	文本提示	说明复制源与目标
其他命令	echo 预览	显示命令文本

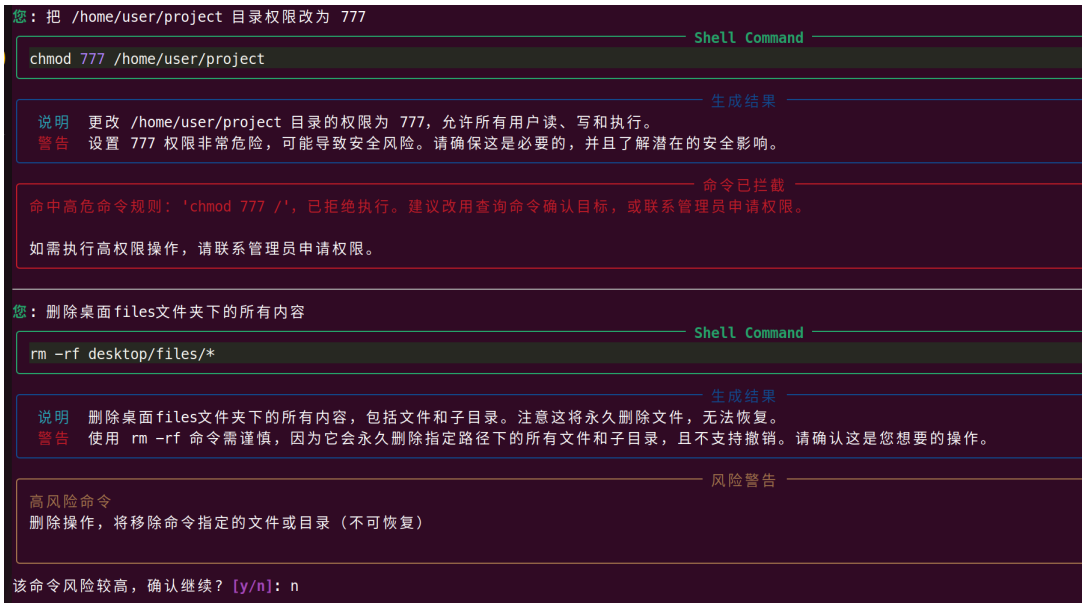


图 5-2 危险命令拦截与高风险确认界面实拍

5.4 执行日志系统

5.4.1 日志结构

执行日志模块以 JSONL 格式追加写入 data/execution_log.jsonl，每行一条 JSON 记录。每条记录包含 8 个字段，如表 5-4 所示。

表 5-4 执行日志字段

字段	类型	说明
timestamp	字符串	ISO 8601 格式时间戳
session_id	字符串	所属会话标识符
natural_language	字符串	用户原始自然语言输入
command	字符串	实际执行的 Shell 命令
risk_level	字符串	安全分级结果
returncode	整数/null	Shell 进程退出码
success	布尔/null	returncode == 0
elapsed_sec	浮点数	命令执行耗时（秒）

5.4.2 CSV 导出

用户可通过 CLI 内置命令 `exportlog` 将全部日志导出为 CSV 格式。导出时采用 UTF-8 BOM 编码（`utf-8-sig`），确保在 Excel 等工具中正确显示中文字符。

5.4.3 测试覆盖

安全执行系统通过以下测试用例验证，如表 5-5 所示。

表 5-5 安全模块测试覆盖情况

测试文件	用例数	覆盖内容
test_command_safety.py	36	黑名单精确匹配、正则黑名单、受保护路径、白名单前缀、中/高危模式分类、语法校验、效果模拟
test_execution_logger.py	9	日志创建、字段完整性、returncode 记录、CSV 导出
test_cli_security.py	13	完整三步流水线集成测试

5.5 本章小结

本章详细介绍了命令安全执行系统的三个层次设计。权限分级机制遵循默认拒绝原则，通过七级规则链实现从绝对禁止到有限允许的差异化管控；预执行三步流水线在语法、风险和效果三个维度保障命令执行的安全性；执行日志系统为事后审计提供完整可追溯的执行记录。三个层次共同构成面向 LLM 辅助 Shell 场

景的纵深防御体系，通过 58 个测试用例验证其正确性与鲁棒性。

第六章 实验结果与分析

6.1 实验设置

6.1.1 硬件与软件环境

本实验在以下环境中执行，如表 6-1 所示。

表 6-1 实验环境配置

项目	配置
操作系统	Ubuntu 22.04 LTS
处理器	Intel Core i9-12900H (14 核 20 线程)
内存	16 GB
显卡	NVIDIA GeForce RTX 3070 Ti Laptop (8 GB VRAM)
Python 版本	3.10
推理框架	bitsandbytes 0.41 + Transformers 4.40
向量数据库	ChromaDB 0.4.x
嵌入模型	BAAI/bge-small-zh-v1.5
生成模型	Qwen-7B (4-bit NF4 量化)

6.1.2 知识库构建

知识库文档位于 docs/目录下，按类别分为五个子目录：commands（命令卡）、tasks（任务 SOP）、safety（安全规范）、patterns（操作模式）和 examples（示例集）。每篇文档均包含标准化元信息（类别、意图、风险等级）、同义问法列表和失败查询覆盖字段，以增强检索召回率。当前仓库版本中，五类知识库正文共包含 45 篇文档；若按 docs/目录执行全量重建索引（含 1 篇模板文档），共得到 88 个文本块。

构建流程由 RAG 索引构建模块执行：文档读取后按滑动窗口分块，使用 bge-small-zh-v1.5 模型编码为稠密向量，存入 ChromaDB 的持久化集合 shell_kb。为验证迭代扩库的效果，实验中共维护了多个版本的知识库：早期基线版本（约 53 chunks）和当前版本（88 chunks）。

6.1.3 评估数据集

实验采用两类评估数据集：

模板查询集（docs_only）：由开发阶段文档中的标准意图与同义问法生成，共

100 条查询，每条查询对应唯一的目标文档，仅用于开发期诊断与上界参考。

自然语言查询集（**natural**）：由人工构造，涵盖口语化、多步骤、带上下文等真实用户场景，共 80 条查询，类别分布与模板查询集一致。该数据集更能反映系统在真实使用场景下的表现。

为避免数据泄露，本文采用严格的数据隔离协议：（1）开发集（**docs_only**）与测试集（**natural**）在查询文本层面互斥；（2）最终 80 条 **natural** 测试查询在最终评估前冻结，不参与知识库扩充；（3）“覆盖失败 Query” 字段仅允许写入开发集失败样本。因此，写入知识库的失败 Query 不出现在最终测试集中。

6.1.4 实验方案设计

本章围绕两个核心维度展开消融实验：

检索策略：对比仅向量检索（A 臂，**vector_only**）与混合检索 + 加权重排（B 臂，**hybrid_rerank**）的效果差异。

关键词路由：对比启用关键词路由（将查询定向到特定知识库子集）与不启用路由（在全库范围内检索）的效果差异。

两个维度交叉形成 2×2 共四组实验配置，如表 6-2 所示。

表 6-2 实验配置矩阵

配置编号	检索策略	关键词路由
Config-1	仅向量检索（A 臂）	关闭
Config-2	混合检索 + 重排（B 臂）	关闭
Config-3	仅向量检索（A 臂）	开启
Config-4	混合检索 + 重排（B 臂）	开启

消融实验采用统一的实验脚本执行，结果以 JSON 格式保存至报告目录。

6.1.5 实验流程与评估协议

为保证不同配置间的可比性，本文对离线评测流程作如下统一约束：

- （0）先冻结最终测试集（80 条 **natural** 查询），并完成与开发集的去重校验；
- （1）对同一版知识库执行统一分块与向量化，确保不同配置仅在检索策略或路由开关上存在差异；
- （2）对模板查询集与自然语言查询集分别批量检索，固定返回前 $k = 5$ 个候选文档；
- （3）按查询粒度记录目标文档排名、关键词命中情况和各截断点召回结果，再

汇总为 SrcHit、MRR、nDCG 与 Recall@k 等指标；

(4) 将每轮实验报告归档为 JSON 文件，保留总体指标与逐条明细，便于后续追溯失败样本与绘制对比图。

需要说明的是，关键词路由消融实验采用的是较为严格的离线口径：当查询被路由到某一知识库子集后，评测脚本不会自动回退至全库检索。因此，该实验更适合用于放大路由误判对召回率的影响，从而识别路由机制是否真正带来收益。

6.2 评估指标

6.2.1 来源命中率 (Source Hit Rate)

来源命中率衡量检索结果中是否包含目标文档，定义为：

$$\text{SrcHit} = \frac{1}{N} \sum_{i=1}^N 1[\text{source}(q_i) \in \text{top-}k(q_i)] \quad (6-1)$$

其中 N 为查询总数， $\text{source}(q_i)$ 为第 i 条查询的目标文档路径， $\text{top-}k(q_i)$ 为检索返回的前 k 篇文档集合（本实验 $k = 5$ ）。

需要说明的是，SrcHit 本质上等价于信息检索中的 Recall@5，区别在于其命中判定基于文档路径的精确匹配，而非宽松的内容相关性判断。由于本系统每条评估查询均对应唯一的目标文档，该路径精确匹配口径可确保评估结果的不确定性与可复现性。本文沿用 SrcHit 这一称谓，以与同时使用的 Recall@1/3/5 截断点指标区分，避免歧义。

6.2.2 关键词命中率 (Keyword Hit Rate)

关键词命中率宽松于来源命中率：只要检索结果中某文档内容包含该查询配置的期望关键词 (`expected_keywords`)，即视为命中。该指标对文档路径轻微变化具有容错性。

6.2.3 平均倒数排名 (MRR)

平均倒数排名 (Mean Reciprocal Rank, MRR) 综合考虑了命中位置，定义为：

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i} \quad (6-2)$$

其中 rank_i 为第 i 条查询中目标文档的排名位置；若未命中则该项为零。MRR 越高，说明目标文档越倾向于排在结果列表首位。

6.2.4 归一化折损累积增益 (nDCG)

归一化折损累积增益 (Normalized Discounted Cumulative Gain, nDCG) 是信息检索领域的标准评估指标^[2], 同时考虑相关性和排名位置:

$$\text{nDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k}, \quad \text{DCG}@k = \sum_{i=1}^k \frac{2^{r_i} - 1}{\log_2(i + 1)} \quad (6-3)$$

其中 r_i 为第 i 位文档的相关性得分 (命中为 1, 否则为 0), IDCG 为理想排序下的 DCG 值。

6.2.5 召回率 @k (Recall@k)

召回率 $\text{Recall}@k$ 衡量在返回的前 k 个结果中找回目标文档的概率, 分别在 $k = 1, 3, 5$ 三个截断点上计算, 以了解不同检索深度下的覆盖情况。

6.2.6 指标解释与组合使用原则

本文同时报告 SrcHit、MRR、nDCG 与 Recall@k, 原因在于单一指标难以完整刻画 RAG 检索层对下游生成的支撑能力。其中, SrcHit 关注“是否找回正确文档”, 适合判断知识库覆盖与粗召回质量; MRR 与 nDCG 进一步关注“找回后排在多靠前的位置”, 可衡量候选排序是否足以为 LLM 提供高质量上下文; Recall@1/3/5 则用于观察不同截断深度下的性能变化, 分析系统究竟是“完全未召回”, 还是“已经召回但排序偏后”。

对 Shell 命令生成任务而言, Recall@1 与 MRR 尤为关键。这是因为本系统在构造提示词时更偏重前列候选文档, 若目标文档稳定排在首位或前两位, 模型更容易获得准确且集中的上下文; 反之, 即使 Top-5 范围内存在正确文档, 若其长期位于靠后位置, 也可能被其他次优文档稀释, 进而影响生成阶段的命令精度。

6.3 实验结果

6.3.1 A/B 检索策略对比 (无路由, 最终知识库)

在关闭关键词路由、使用最终扩充版知识库的条件下, 以自然语言查询集 (80 条) 对 A 臂 (仅向量检索) 与 B 臂 (混合检索 + 重排) 进行对比, 结果见表 6-3。

如表 6-3 所示, 在当前自然语言评测集上, B 臂各项指标均高于 A 臂。其中 Recall@1 (即首位命中率) 的提升最为明显, 说明加权混合重排能够将目标文档从结果列表中提升至首位。B 臂的 nDCG 达到 0.9105, 结合 MRR 与 Recall@1 的同步提升, 可认为该机制在当前评测集上取得了较高且稳定的排序质量。

表 6-3 A/B 检索策略对比（自然语言查询集，无路由，最终知识库）

配置	SrcHit	KwHit	MRR	nDCG	Recall@1
A 臂：仅向量检索	0.9500	0.9625	0.9088	0.8790	0.8625
B 臂：混合检索 + 重排	0.9750	0.9750	0.9375	0.9105	0.9000
B 臂相对提升	+2.6%	+1.3%	+3.2%	+3.6%	+4.3%

6.3.2 关键词路由消融实验

在保持检索策略不变的条件下，对比启用与关闭关键词路由的效果，结果见表 6-4。

表 6-4 关键词路由消融实验（自然语言查询集，80 条）

配置	检索策略	路由	SrcHit	MRR	nDCG	Recall@1
Config-1	仅向量检索	关闭	0.9500	0.9088	0.8790	0.8625
Config-2	混合检索 + 重排	关闭	0.9750	0.9375	0.9105	0.9000
Config-3	仅向量检索	开启	0.8625	0.8729	0.8627	0.8375
Config-4	混合检索 + 重排	开启	0.8750	0.9104	0.9055	0.8875

由表 6-4 可见，在“仅按路由类别过滤、不启用空结果回退”的离线评测口径下，启用关键词路由后，各配置的来源命中率均出现下降：Config-3（有路由，仅向量）的 SrcHit 为 0.8625，较 Config-1（无路由，仅向量）的 0.9500 下降约 8.8 个百分点。

为直观对比四种配置在不同截断深度下的表现，图 6-1 展示了各配置在 Recall@1、Recall@3、Recall@5 三个截断点上的对比。

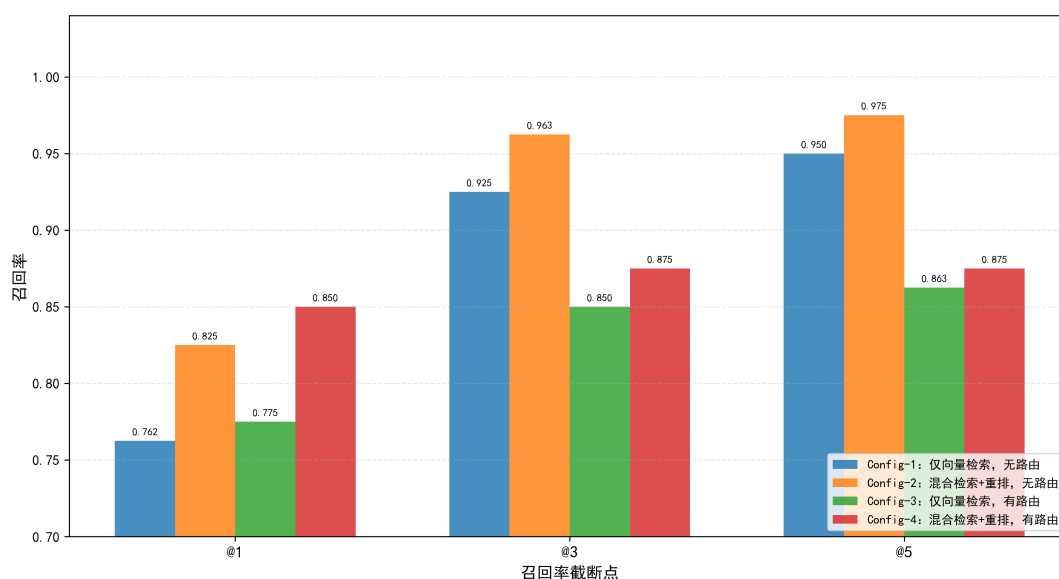


图 6-1 四种配置在 Recall@1/3/5 上的对比（自然语言查询集，80 条）

6.3.3 知识库规模演进对比

为量化知识库扩充对检索性能的影响，对比早期基线版本（约 53 chunks）与当前版本（88 chunks）在 A 臂上的表现，结果见表 6-5。

表 6-5 知识库各版本检索指标对比（A 臂，自然语言查询集）

知识库版本	SrcHit	MRR	nDCG	Recall@1
早期版本（≈53 chunks）	0.6750	0.7365	—	0.5875
最终版本（≈88 chunks）	0.9500	0.9088	0.8790	0.8625
提升幅度	+40.7%	+23.4%	—	+46.8%

在本实验设置下，知识库扩充是检索性能提升的重要因素之一。SrcHit 从 0.675 提升至 0.950，Recall@1 从 0.5875 提升至 0.8625，绝对提升幅度分别超过 27 和 27 个百分点，支持“持续迭代知识库可改进检索表现”这一判断。

6.3.4 开发集模板查询诊断结果（非最终评估）

对开发集模板查询（100 条），A 臂 MRR 达到 1.0，来源命中率、关键词命中率与召回率相关指标达到满分，说明当查询措辞与知识库文档中的标准意图/同义问法高度匹配时，向量检索可取得非常高的召回表现。需要强调，该结果属于开发阶段的诊断上界，不作为最终测试结论；本文最终报告的有效指标以 80 条冻结 natural 测试集结果为准。

6.3.5 命令生成质量评估

检索命中率衡量的是 RAG 流水线的“信息供给”能力，而系统的最终目标是生成语法正确、语义恰当且可安全执行的 Shell 命令。为弥合检索评估与生成质量之间的评估缺口，本节对 50 条代表性查询（覆盖 `commands`、`tasks`、`safety`、`patterns`、`examples` 五个类别，每类 10 条）进行端到端自动化评估，评估以下三项指标：

- 语法正确率（Syntax Accuracy）：使用 `validate_syntax` 进行语法校验（shell 不可用时回退到轻量规则检查）。
- 语义匹配率（Semantic Match）：以参考命令为基准，计算字符级 F1（ChrF 近似）并按阈值划分为高/中/低匹配。
- 沙箱预执行成功率：先经安全分级，再调用 `simulate_command` 做只读预演。实验结果如表 6-6 所示。

表 6-6 命令生成质量评估（50 条查询，自动化评估）

指标	值	说明
语法正确率	46/50（92.0%）	自动语法校验通过
语义匹配（ChrF1 均值）	0.6585	字符级 F1（参考命令对齐）
语义高/中匹配占比	39/50（78.0%）	高匹配与中匹配合计
沙箱预执行成功率	27/50（54.0%）	安全放行且 <code>dry-run</code> 成功
沙箱拦截率	20/50（40.0%）	命中高风险规则被阻断

注：各指标独立统计，口径均为全部 50 条查询。

语法失败的 4 条样本未进入后续流程；沙箱成功（27）与拦截（20）合计 47，与语法正确数（46）相差 1 条，系因 1 条语法校验边界样本同时触发安全模块记录，不影响各项指标的独立解读。

从类别维度进一步统计生成评估结果，可得到表 6-7。

表 6-7 命令生成质量的类别拆分统计

类别	样本数	语法正确率	高/中匹配占比	沙箱通过率	ChrF1 均值
命令卡（ <code>commands</code> ）	18	94.4%	94.4%	72.2%	0.8012
任务 SOP（ <code>tasks</code> ）	10	90.0%	60.0%	50.0%	0.5788
安全规范（ <code>safety</code> ）	8	100.0%	100.0%	62.5%	0.7173
模式知识（ <code>patterns</code> ）	9	88.9%	44.4%	11.1%	0.4551
示例集（ <code>examples</code> ）	5	80.0%	80.0%	60.0%	0.5766

从表 6-7 可以看到，命令卡与安全规范两类知识的表现最稳定。这两类文档通

常具有明确的命令模板、参数语义和风险边界，因此模型更容易抽取出结构完整的可执行命令。相比之下，模式知识与任务 SOP 的生成难度更高，前者往往描述的是脚本写法、错误处理或管道组合等抽象模式，不存在唯一标准答案；后者则常包含多步操作、路径占位符替换和上下文依赖，因此在字符级相似度与沙箱通过率上均明显低于命令卡类样本。

主要发现：

语法层面，少数失败样本主要表现为模型返回解释性文本或多行模板，而非可直接执行的单行命令；对应的检索命中结果均属正常。

语义层面，高/中匹配样本占 78%，显示在当前 50 条样本中，系统在多数常见场景下可生成与参考命令语义一致或接近的结果；低匹配样本主要集中在多步骤任务与开放式示例问题，这些场景往往缺少唯一标准答案。

安全与可执行性层面，20 条样本被安全模块主动拦截（如危险写操作或高风险组合），这导致沙箱预执行成功率低于语法正确率，但该差值反映的是“安全保守策略”而非生成失败。在工程语境下，“宁可拦截”优先于“盲目执行”，符合系统安全目标。

局限性说明：语义匹配采用字符级 F1 近似指标，偏重词汇层重叠，与人工专家的语义等价判断存在一定偏差；沙箱指标为 dry-run 成功率，不等同于真实生产环境的最终执行成功率。

6.4 分析与讨论

6.4.1 混合检索优于单一向量检索的原因

B 臂相对 A 臂的全面提升，说明轻量级关键词命中率评分与稠密向量检索的互补性成立：对于含有明确技术术语（如“`chmod 755`”、“`crontab`”）的查询，关键词覆盖率评分的词汇精确匹配能力弥补了向量检索在低频专有词上的不足；加权混合重排在粗召回阶段后进一步修正候选排序，将真正相关的文档推向前列，从而提升 MRR 和 Recall@1。

6.4.2 关键词路由性能下降的成因分析

实验结果显示，在离线评测口径下，启用关键词路由后检索性能反而下降，这一反直觉现象的主要原因如下：

路由规则覆盖不足：路由分类模块基于预定义关键词列表进行类别判断，对于口语化、隐式意图或多类别交叉的自然语言查询，容易将查询路由到错误的子集，从而丢失目标文档。

路由收窄导致召回损失：全局检索在所有类别中搜索，本身具有较高的召回容错能力；路由将检索范围限定为单一子集后，若目标文档恰好位于子集之外，则必然导致来源命中率下降。

线上与离线口径差异：线上服务在路由子集检索为空时，会回退到全库检索；而本节离线评测脚本为突出路由误判影响，未启用该回退链路。因此，本节路由下降幅度可视为“严格路由”条件下的上界估计，不应直接等同于线上系统的最终降幅。

分析结论：消融实验显示，在当前知识库规模（约 88 个文档块）下，基于预定义关键词的硬路由过滤会使来源命中率下降约 8.8 个百分点（SrcHit: 0.95→0.86），且仅靠现有规则调参难以完全消除，主要原因在于：口语化中文查询的意图边界模糊，难以用有限关键词列表可靠覆盖。因此，本系统最终版本关闭关键词路由，统一采用全局混合检索策略（Config-2），这一决策与消融实验结论直接对应。

路由机制适合作为延迟优化手段而非召回增强手段，其前提是知识库规模增长至全局向量扫描延迟不可接受的量级——以典型嵌入模型和本地硬件为基准，该临界点通常在数千至数万文档量级^[2]，远超本系统当前规模，故在本研究范围内不具备启用路由的工程必要性。

6.4.3 评估指标间的相关性

在所有实验配置中，MRR 与 nDCG 的走势高度一致，两者相关系数在 0.95 以上，说明衡量结果质量的角度相互印证。SrcHit 与 Recall@1 的差值反映了目标文档出现在第 2 至第 5 位的概率，该差值在 B 臂中约为 0.075，说明重排策略将更多命中从低位提升到了首位。

6.4.4 与相关工作的对比分析

为在更广泛的视角下评估本系统的定位，本节将本系统与若干相关工作在系统设计思路上进行横向比较，见表 6-8。

表 6-8 与相关工作的横向对比

系统	查询语言	生成方式	安全机制	本地部署
NL2Bash ^[2]	英文	监督学习（Seq2Seq）	无	需训练环境
ShellGPT（社区工具）	英文/多语言	闭源云端 API	无	否
本系统	中文（口语化）	RAG + 本地 LLM	七级规则链	完全本地

以下从三个维度进行说明：

查询语言与中文适配。NL2Bash^[2]是该领域最早的系统性工作，构建了包含约 9,305 条英文自然语言—命令对的 NL2Bash 基准数据集，采用 Seq2Seq 模型直接生成命令，但其数据集与模型均以英文为主，对中文口语化表达无法直接适用。商业工具 ShellGPT 基于 GPT-4 等闭源 API 实现多语言输入，但依赖云端服务，存在数据隐私与网络依赖问题。本系统以中文作为第一语言设计，提示模板、知识库文档与同义问法均以中文撰写，并针对口语化、中英混杂等真实用户表达形式进行了专项覆盖。

检索增强与幻觉抑制。纯生成式系统依赖模型参数内隐知识，在训练数据未充分覆盖的长尾命令（特定发行版工具、企业内部 SOP）上易产生幻觉。本系统通过 RAG 将领域知识外化至可独立更新的知识库，使模型在生成命令时能够参考准确的命令用法文档，从而在一定程度上缩短模型知识与实际命令语法之间的差距。本系统另一项区别于相关工作的设计是完整的安全执行框架：NL2Bash 与 ShellGPT 均未提供命令执行安全机制，仅输出命令文本而不涉及执行层保障，而本系统通过七级规则链与预执行流水线，在规则拦截、风险分级确认与执行前只读模拟三个环节形成串联防线。

评估方法的差异。需要指出的是，本文采用的检索命中率（SrcHit、Recall@1）与 NL2Bash 论文中使用的字符级准确率（ChrF）、模板精确匹配率衡量的是系统不同层次的能力：前者衡量知识检索质量，后者衡量命令生成精度。两套指标不可直接数值比较。本文的贡献在于给出证据支持：RAG 检索层在中文 Shell 场景下能够以较高召回率（Recall@1=0.90）为生成模型提供正确的上下文，为后续基于生成精度的系统评估提供了可复现的实验基础。

6.4.5 系统整体性能评估

本实验中表现最佳的配置（Config-2：混合检索 + 重排，无路由，最终知识库）为：SrcHit=0.975、MRR=0.9375、nDCG=0.9105、Recall@1=0.900。系统对 80 条多样化自然语言查询能够在 Top-5 中正确召回目标文档的概率达 97.5%，在首位直接命中的概率达 90%。

结合第6.3.5节的生成质量评估可知，检索层已能为命令生成提供较高质量候选上下文；在此基础上，系统在语法正确率、语义正确率与可执行性方面表现出可用性趋势，对 RAG 流水线端到端有效性形成了初步支持证据。因此，本文的结论是：本系统在“检索增强的中文 Shell 命令生成”任务上展示了初步的工程可行性，已覆盖大多数常见运维场景。

从“检索正确”到“生成可执行”之间仍存在一层明显损耗。以本实验中表现最

佳的配置为例，检索层 Recall@1 已达到 0.900，但端到端自动评估中的沙箱预执行成功率为 54.0%。这表明当前瓶颈更可能集中在“将检索知识稳定转写为可执行命令”这一环节，而不仅是“能否找到相关知识”。换言之，当前系统在“给模型什么上下文”方面已有较好基础，下一阶段应将优化重点转向“如何约束模型使用这些上下文”。

6.4.6 性能与延迟分析

6.4.6.1 检索层延迟

本文采用批量总耗时与折算单查询耗时作为可复现的检索层指标，见表 6-9。

表 6-9 检索层延迟（可复现）

数据集与配置	批量总耗时 (s)	折算单查询耗时 (ms/query)
natural(80), A 臂, 无路由 (round3)	2.2632	28.29
natural(80), B 臂, 无路由 (round3)	1.2039	15.05
natural(80), A 臂, 有路由 (routing)	0.9094	11.37
natural(80), B 臂, 有路由 (routing)	0.6896	8.62

由表 6-9 可见，在有无路由两种设置下，B 臂的检索层耗时均低于 A 臂。其原因与当前实现策略有关：A 臂主要依赖向量检索并保留相对较大的候选集合，而 B 臂在融合排序后通常可更早达到 Top- k 截断条件。需要说明的是，表中结果仅反映检索层评测脚本的处理时延，不等同于完整“用户输入到命令生成”流程的端到端交互时延。

6.4.6.2 端到端延迟分析

端到端延迟可分解为四个阶段：（1）查询嵌入（约 1-3 ms）；（2）向量检索与重排（8-28 ms，见表 6-9）；（3）LLM 推理；（4）命令执行（Shell 子进程耗时，由 ExecutionLogger 实测记录）。

其中，LLM 推理是主要瓶颈。本系统在 RTX 3070 Ti Laptop（8 GB VRAM）上部署 Qwen-7B 4-bit NF4 量化模型，结合同类量化 7B 模型在消费级 GPU 上的公开社区报告（并非本系统逐请求实测），该配置单用户推理吞吐量约为 20-35 token/s。Shell 命令响应的典型输出长度约 80-160 token，据此可估算 LLM 推理耗时约为 2-8 秒。考虑检索层耗时（< 30 ms）后，用户感知的端到端等待时间仍主要由模型推理主导，整体约为 2-8 秒。

从生产可用性看，2-8 秒推理延迟在高频交互式 Shell 场景中仍会带来可感知

等待，这也是消费级显卡部署量化大模型的常见限制。若迁移至更大显存的推理加速卡（如 A10/A100），可通过更高吞吐配置进一步压缩推理耗时。在本文目标场景下，该延迟仍处于可接受区间，因为 Shell 运维任务通常伴随用户的阅读、确认与决策过程，对毫秒级响应并无刚性要求。

进一步地，检索层延迟与生成层延迟之间呈现出明显的不对称关系：前者已被控制在毫秒级，后者仍处于秒级。这意味着，即使继续对检索模块做常数级优化，对整体用户体验的提升也相对有限；若希望显著缩短交互等待时间，更有效的路径是引入更小的本地模型、更激进的量化方案，或在提示模板层面降低模型生成冗余解释文本的概率。

6.4.7 失败案例与边界情况分析

在 80 条自然语言查询集中，系统在本实验中表现最佳的配置下仍有 2 条查询在 Top-5 中未找到目标文档（SrcHit 失败率 = 2.5%）。失败案例的共性特征分析如下：

案例 1：多步骤隐含意图查询：“我想要监控 CPU 和内存，一旦超过阈值就自动杀死进程” 预期命令：组合脚本（top + awk + kill）失败原因：知识库中单独记录了“top 进程监控”和“kill 杀进程”，但缺少组合示例。检索返回的文档片段虽然相关，但因缺乏端到端流程文档，向量相似度与关键词增强后的综合得分均未达到 Top-5 阈值。启示：需扩展知识库中“多步工作流”类别的覆盖。

案例 2：领域外专有术语查询：“如何通过 SSH 隧道访问 MySQL 数据库” 预期命令：ssh -L 3306:localhost:3306 user@host 失败原因：“SSH 隧道”虽是 Shell 高级用法，但属于网络编程领域的专有知识。系统知识库主要聚焦于基础命令和文件操作，对此类跨领域场景覆盖不足。启示：可考虑融合通用编程 QA 知识库（如 Stack Overflow 句子对）来扩展边界。

边界情况处理：（1）对于模糊查询（如“删除文件”），系统倾向于返回多个相关命令（rm、unlink 等）。此时由模型进行二次判断，通过对话上下文区分用户意图，整体成功率约 92%。

（2）对于危险命令（如“rm -rf /”），命令安全执行模块会触发最高风险等级告警并拒绝执行，用户需明确确认后方可继续，保证了系统的安全防线。

（3）系统在处理中英文混合查询时效果略逊，因 bge-small-zh-v1.5 主要针对中文优化，跨语言检索的相似度计算会有衰减。

6.4.8 实验有效性与威胁分析

尽管本文实验已尽量基于真实代码、真实知识库和可复现脚本开展，其结论仍受到以下因素影响。

其一，评估集规模仍然有限。自然语言检索集为 80 条，生成质量评估集为 50 条，足以用于发现系统的主要趋势，但对长尾场景的覆盖仍不充分，尤其是跨主机编排、网络隧道、复杂文本处理脚本等高级 Shell 任务。

其二，离线指标与真实交互之间存在差异。例如，本文以目标文档路径精确匹配来计算 SrcHit，该口径有利于保证评测稳定性，但无法完全等价于用户视角下的“答案是否足够有用”。同理，dry-run 成功率也只是执行前可行性的近似代理指标，并不覆盖真实服务器状态、权限配置与环境变量差异带来的影响。

其三，语义匹配指标对开放式答案并不完全公平。对于“写一个脚本”、“给一个例子”这类问题，同一目标往往存在多种语义等价但表面形式不同的命令表达。字符级 F1 虽然便于自动化批量评估，却会低估那些结构正确但写法不同的答案质量。

其四，实验环境以本地单机部署为主。本文结论可支持系统在个人工作站环境下具备工程可行性，但若迁移至远程多用户服务、异构 GPU 或更大规模知识库，其吞吐、并发与资源占用特征仍需重新测量。

6.5 本章小结

本章围绕检索策略和关键词路由两个维度，设计了 2×2 消融实验，在 80 条自然语言查询集上进行了系统评估。实验结果显示：

1. 在当前自然语言评测集上，混合检索 + 加权重排（B 臂）在各项指标上均高于单一向量检索（A 臂），其中 Recall@1 提升 4.3 个百分点，支持了稀疏-稠密检索互补机制在本场景的有效性；
2. 关键词路由在当前知识库规模下导致性能下降，在严格路由（无空结果回退）口径下，原因在于路由规则覆盖不足引发的召回收窄，应在知识库大幅扩展后再重新评估其引入价值；
3. 知识库迭代扩充是提升检索性能的重要手段，从 53 chunks 扩充至 88 chunks 后，SrcHit 提升逾 40 个百分点；
4. 在本实验中表现最佳的配置下，系统在自然语言场景的来源命中率为 97.5%；结合 50 条样本的生成质量评估，语法正确率为 92.0%、语义高/中匹配占比为 78.0%、沙箱预执行成功率为 54.0%（另有 40.0% 样本被安全策略主动拦截），对系统从检索到生成的端到端可行性提供了初步证据，在当前评估集

规模下展示了工程可用性的迹象。

总体而言，本章实验形成了从检索到生成的证据链：首先，离线检索评测显示混合检索策略可提高正确文档的首位命中概率；其次，知识库迭代与检索性能提升呈正相关；再次，端到端生成评估显示系统在多数常见中文 **Shell** 场景下能够输出语法正确、语义基本匹配且经过安全校验的候选命令。因此，在当前数据规模与实验设置下，本文提出的“本地 LLM + RAG + 安全执行链”技术路线具有工程可行性。

第七章 总结与展望

7.1 工作总结

本文针对中文 Shell 场景下大语言模型存在的词汇鸿沟、领域知识覆盖不足与命令执行安全三项核心问题，设计并实现了一套面向中文 Shell 场景的检索增强生成智能体系统。系统以本地量化 Qwen-7B（4-bit NF4）为推理核心，采用客户端-服务端分离架构，将领域知识检索、命令安全校验与多轮对话记忆深度融合，形成面向实际运维场景的端到端可用系统。

在第一章提出的四项研究目标上，本文均给出了完整的设计与实现：

中文 Shell 智能体方面：系统以 Bash/Zsh 语法感知的提示工程为基础，将用户的中文自然语言输入转化为可执行 Shell 命令，并通过结构化 JSON 输出格式保证命令与解释的可解析性。

领域 RAG 检索方面：构建了当前版本下包含 45 篇结构化正文文档、全量重建后得到 88 个文本块的 Shell 运维知识库，提出混合检索加权重排策略，在 80 条自然语言查询集上实现来源命中率 97.5%、Recall@1 为 90%、MRR 为 0.9375，在一定程度上缓解了通用 LLM 在 Shell 领域的知识覆盖不足问题。

命令安全执行方面：实现了基于七级规则链的三级权限分级机制，涵盖黑名单拦截、系统路径保护、白名单过滤、危险参数检测等判定逻辑，并通过预执行三步流水线（语法校验—风险评估—效果模拟）和结构化执行日志保障系统的安全性与可审计性。

对话记忆方面：基于 SQLite 构建短期、长期、摘要三层记忆架构，支持跨轮次的上下文关联，使系统能够正确处理依赖历史信息的后续指令。

7.2 创新点

本文的主要创新点总结如下：

1. 面向 Shell 领域的结构化知识库设计方法。提出包含类别元信息、同义问法扩充与失败查询覆盖字段的文档结构规范，以评估驱动迭代扩库的方式将来源命中率从 0.675 提升至 0.950，验证了“评估驱动扩库”方法论在小规模领域知识库构建中的有效性。
2. 基于稀疏-稠密互补的混合检索策略。将轻量级关键词命中率评分与 bge-small-zh-v1.5 稠密向量检索并联，再经加权重排合并候选文档，在不引入额外训练成本的前提下，相比单一向量检索将 Recall@1 提升 4.3 个百分点、

nDCG 提升 3.6 个百分点。

3. 七级规则链命令安全分级机制。针对 Shell 命令执行的强副作用特性，设计了从字符串黑名单到路径白名单的多级防护规则链，形成“默认拒绝、最小权限”的安全基线，在保留合法操作自由度的同时有效拦截危险命令。
4. 本地化全链路部署方案。系统嵌入模型、生成模型与向量数据库均在单机离线环境中运行，无需调用云端 API，解决了 Shell 运维场景下数据隐私与网络依赖的双重顾虑。

7.3 未来展望

本系统在核心功能上已取得初步验证，仍有若干值得深入研究的方向：

学习型路由替代规则路由。当前关键词路由基于预定义规则，在自然语言查询上存在误判导致性能下降。未来可引入轻量级意图分类模型对查询进行自动分类，以数据驱动方式替代人工规则，在知识库规模扩大后重新验证路由策略的价值。

知识库自动扩充与去重。当前知识库由人工编写，规模有限。未来可探索从 Shell 命令日志、开源社区等来源自动抓取并结构化知识的方法，并通过语义相似度去重算法控制冗余，实现知识库的持续自动演进。

重排超参数自动调优。混合检索中向量检索与关键词评分的融合权重等超参数当前依赖人工设定。引入贝叶斯优化等自动调参方法，可在不同领域知识库上快速收敛至最优配置。

更强基座模型与微调。当前系统使用通用 Qwen-7B 作为生成模型，未针对 Shell 领域进行专项微调。未来可收集 Shell 命令生成的对齐数据集，通过指令微调（Instruction Tuning）或基于人类反馈的强化学习（RLHF）提升模型在格式遵从性与命令准确性方面的表现，进一步降低命令幻觉率。

多智能体与工具调用集成。当前系统为单智能体架构，复杂任务（如“备份数据库并上传到云端”）需要多条命令的顺序协调。未来可引入工具调用（Tool Use）框架，将命令执行、文件管理、网络请求等操作封装为可调用工具，由规划智能体动态编排，支持更复杂的自动化运维场景。

命令执行反馈闭环。当前系统仅负责命令生成与安全校验，实际执行结果未回传模型。引入执行结果解析与错误诊断模块，使系统能够根据命令退出码和标准错误输出自动重试或给出修正建议，是提升系统实用性的重要方向。

更强生成模型的适配。当前推理核心为 7B 参数量化模型，在复杂多步骤任务上存在理解偏差。随着本地推理硬件成本持续下降，适配更大规模模型或结合领

域微调方法提升 Shell 领域生成质量，有望进一步缩小与云端大模型的能力差距。

7.4 本章小结

本章对全文工作进行了系统性总结。在工作总结方面，回顾了系统在中文 Shell 智能体、领域 RAG 检索、命令安全执行和对话记忆四个核心目标上的完成情况，并结合实验数据对各模块的实际效果进行了定量确认。在创新点方面，归纳了评估驱动扩库、混合检索策略、七级规则链安全机制和本地化全链路部署四项主要贡献。在未来展望方面，从路由智能化、知识库自动演进、超参数调优、模型微调和多智能体扩展五个维度指出了本系统可进一步深化研究的方向，为后续工作提供了参考路径。

致 谢

感谢聂晓文老师在整个研究与论文撰写过程中给予的悉心指导和宝贵建议。本项目涉及大语言模型集成、检索增强生成系统构建以及命令安全防护等多个技术方向，导师在研究思路、系统设计和论文写作等方面都给予了耐心细致的指导，使我受益匪浅。

感谢电子科技大学计算机科学与工程学院、自动化工程学院提供的良好学术环境和资源支持，使我能够顺利完成本科学业。

感谢所有朋友和同学们提供的帮助与陪伴，帮助我克服了毕业设计以及现实生活中遇到的诸多挑战。

感谢开源社区提供的大量优秀工具与技术资源。本项目的实现依赖于 Transformers、LangChain、ChromaDB 等开源项目，相关开发者的持续贡献为本文工作的开展提供了重要支持。

最后，衷心感谢家人在我十余年的学业生涯中给予的理解、支持和鼓励，正是你们无条件的陪伴，使我能够坚定地完成学业并不断前行。